

An finite element analysis surrogate model with boundary oriented graph embedding approach for rapid design

Xingyu Fu¹, Fengfeng Zhou¹, Dheeraj Peddireddy², Zhengyang Kang³, Martin Byung-Guk Jun^{1,4,*} and Vaneet Aggarwal²

¹School of Mechanical Engineering, Purdue University, West Lafayette, IN 47906, USA

²School of Industrial Engineering, Purdue University, West Lafayette, IN 47906, USA

³School of Mechanical and Power Engineering, Nanjing Tech University, Nanjing, Jiangsu 211800, China

⁴Indiana Manufacturing Competitiveness Center (IN-MaC), West Lafayette, IN 47906, USA

*Corresponding author. E-mail: mbgjun@purdue.edu

Abstract

In this work, we present a boundary oriented graph embedding (BOGE) approach for the graph neural network to assist in rapid design and digital prototyping. The cantilever beam problem has been solved as an example to validate its potential of providing physical field results and optimized designs using only 10 ms. Providing shortcuts for both boundary elements and local neighbor elements, the BOGE approach can embed unstructured mesh elements into the graph and performs an efficient regression on large-scale triangular-mesh-based finite element analysis (FEA) results, which cannot be realized by other machine-learning-based surrogate methods. It has the potential to serve as a surrogate model for other boundary value problems. Focusing on the cantilever beam problem, the BOGE approach with 3-layer DeepGCN model achieves the regression with mean square error (MSE) of 0.011 706 (2.41% mean absolute percentage error) for stress field prediction and 0.002 735 MSE (with 1.58% elements having error larger than 0.01) for topological optimization. The overall concept of the BOGE approach paves the way for a general and efficient deep-learning-based FEA simulator that will benefit both industry and Computer Aided Design (CAD) design-related areas.

Keywords: machine learning, graph neural network, stress field, solid mechanics, topology optimization

1. Introduction

Finite element analysis (FEA), which plays a critical role in modern engineering computational design and digital prototyping, provides numerical solutions for boundary value problems and simulates the distribution of physical fields with high precision (Axelsson & Barker, 2001; Tepjit et al., 2019). The engineers or the designers can refer to the computed FEA result to evaluate the quality of the design, and based on these physical field results, some automatic algorithms can assist in optimizing the design work (e.g., topological optimization in solid mechanics). However, FEA generally requires high-performance computational resources and has an expensive time cost which becomes a bottleneck in the present manufacturing industry, especially for advanced time-sensitive techniques: inverse modeling, agile manufacturing, generative design system, rapid prototyping, etc. (Courtecuisse et al., 2014; Khadilkar et al., 2019; Nie et al., 2021). In order to improve the efficiency of this computational design process, some researchers employ forward interpolation techniques (e.g., proper orthogonal decomposition; Roy et al., 2021) to reduce the finite element model to largely accelerate the calculation time with much fewer computation resources. Others use machine learning technologies to fit the distribution of physical fields to obtain instant simulation results with compromising accuracy. Compared with these forward interpolation techniques, the machine learning model has some unique benefits for the application. First,

though the training of the machine learning model takes quantities of time and computation resources, the network prediction usually does not require high-end hardware support, which can be easily deployed on portable devices and cost-efficient edge devices. This can benefit the customized services in Industry 4.0 and can largely expand the application scope due to the low requirement of the algorithm deployment. Further, the network prediction usually can be executed on a millisecond scale regardless of the size of the input model as long as it is considered in the dataset, which assists to provide stable computation services. Last, the main merit of the machining learning model is that it has the ability to solve abstract decision-making problems. The objective of the FEM is to assist humans to optimize the design while the machine learning model has the potential to provide an end-to-end solution to directly output the optimized design (Banga et al., 2018; Lee et al., 2020, 2022; Nie et al., 2021; Zhang et al., 2019), which can largely save humans' mental labor. Therefore, the machine learning method is employed in this paper to interpolate the FEM computation results. The main objective of this paper is to use the machine learning approach to provide an effective method for solving boundary value problems to improve the efficiency of the digital design process and investigate the potential of this method to directly output the optimized design according to the physical field information.

Received: October 11, 2022. Revised: February 7, 2023. Accepted: February 8, 2023

© The Author(s) 2023. Published by Oxford University Press on behalf of the Society for Computational Design and Engineering. This is an Open Access article distributed under the terms of the Creative Commons Attribution License (<https://creativecommons.org/licenses/by/4.0/>), which permits unrestricted reuse, distribution, and reproduction in any medium, provided the original work is properly cited.

Some researchers employed conventional machine learning methods such as support vector regression (Capuano & Rimoli, 2019) and artificial neural network (Tamaddon-Jahromi et al., 2020) to approximate the physical field distribution and solve the boundary value problem. However, most methods are for specific boundary conditions and require extra training ahead of the application. This limits their capability of serving as a general FEA surrogate model, and their ability to solve abstract decision-making problems. Also, these methods usually cannot directly provide an end-to-end design solution.

Recently, with the rapid development of deep learning (DL) techniques, some researchers employed convolutional neural network (CNN) as the backbone for solving physical field prediction problems, and found out that CNN-based surrogate models cannot only solve basic boundary value problems [e.g., solid mechanics (Kantzios et al., 2019; Khadilkar et al., 2019; Nie et al., 2020), fluid dynamics (Guo et al., 2016; Kutz, 2017; Zhang et al., 2018a), etc.] but also perform efficiently on some abstract design problems based on predicted physical field [e.g., topology optimization (Banga et al., 2018; Lee et al., 2020; Nie et al., 2021; Zhang et al., 2019)]. However, CNN-based surrogate models still have inevitable disadvantages. Firstly, CNN can only accept input tensors with fixed input sizes, which becomes an obstacle for input tensor encoding, especially for some complex FEA conditions. Also, since CNN can only perform calculations on Euclidean coordinates, most CNN-based surrogate models employ square- or cube-shaped meshes [two-dimensional (2D) quadrilateral or 3D hexahedron meshes with uniform side lengths] that require homogeneous grid generation on the input FEA model. The grid resolution sometimes cannot reach a sufficiently high level since the CNN usually cannot handle a large tensor input with hundreds of meshes along X/Y/Z axes due to the limitation of modern computation resources. Lastly, most FEA runs on unstructured meshes (e.g., triangular meshes or polygon meshes with different side lengths) that cannot directly fit into the CNN input tensor. Some researchers have presented alternate solutions to handle this problem, which is illustrated in Fig. 1. We provide a von Mises stress distribution prediction for a 2D cantilever beam based on simulation results from ABAQUS (Abaqus, 2011). The beam is made of A36 steel with Young's modulus 200 GPa and Poisson's ratio of 0.32 and has an elliptical hole inside. Its left edge is fixed and an external force of 1000 N is exerted on the center of its right edge. The ground truth simulation result with triangular meshes (with an approximate global mesh size of 1.0 mm) is shown in Fig. 1b. One type of method for using CNN to regress the physical field is to approximate the original input model's geometry with a high-resolution voxelized representation, shown in Fig. 1c (Nie et al., 2020). However, the resolution is still limited by computational resources, and the simulation result can significantly change since the change in model shape can largely influence the global boundary condition, shown in Fig. 1c. The other type of method usually uses some post-process tasks, e.g., averaging the physical fields to fit into the square-/cube-shaped meshes. This will still lead to an extra approximation error for the output results, shown in Fig. 1d. All of those defects call for a more efficient DL surrogate model for FEA and design optimization. The graph neural network (GNN) comes into sight.

The GNN is a generalized version of CNN for non-Euclidean graphs, which has been widely applied to abstract decision-making and data-regression problems on different graph data structures, e.g., point cloud, triangular meshes, and molecular structures (Alet et al., 2019; Belbute-Peres et al., 2020; Gilmer et al., 2017; Guo & Buehler, 2020; Ogoke et al., 2020; Pfaff et al., 2020; Sanchez-Gonzalez et al., 2020; Wang et al., 2020). Since the unstruc-

tured meshes can be naturally embedded by an undirected graph G , many researchers investigated GNN-based surrogate models to solve FEA regression problems, as shown in Table 1. It is assumed that an undirected graph $G = (V, E)$ represents the embedded unstructured meshes in an FEA model. Then, generally, $V = 1, \dots, N$ can be the set of N graph vertices (or graph nodes) denoting N geometrical node points in FEA while $E \subseteq V \times V$ is the set of graph links (or graph edges) representing the connection of neighborhood geometrical nodes. Here, since the graph theory also has the concept of the nodes and edges, in order to make it different from mesh nodes and edges in FEA, we use "graph vertex" and "graph link" to refer to the related concepts in graph theory, and we use "mesh/element node", "mesh/element edge", and "mesh/element body" to refer to the concepts of from FEA. Graph vertex v contains features f_v encoding the properties of a geometrical node point (e.g., coordinates, material properties, and boundary conditions). Graph links generally employ adjacent matrix $E \in [0, 1]^{N \times N}$ to describe the connection between geometrical node points. Nowadays, most applications employed GNN layers with message passing neural network (MPNN) framework (Gilmer et al., 2017) whose updated message m_v^{t+1} of graph vertex v at layer $t + 1$ can be represented by

$$m_v^{t+1} = \sum_{\omega \in N(v)} M_t((h_v^t, h_\omega^t, e_{v\omega})) \quad (1)$$

$$h_v^{t+1} = U_t((h_v^t, m_v^{t+1})) \quad (2)$$

$$\hat{p} = R(\{(h_v^T | v \in G)\}) \quad (3)$$

where M_t , U_t , and R are the message function, vertex update function, and readout function, respectively; h represents the hidden state in GNN layers (e.g., h_v^t is the hidden state of v at layer t); $N(v)$ denotes the neighbors of v in graph G ; $e_{v\omega}$ represents the graph link features; and $\hat{p}$ is the final prediction result. From equation (2), it can be seen that GNN layers can only pass the information of the embedded feature to its neighbor vertices. If the distance on a graph for passing an important message is relatively long, with current graph embedding approaches, more layers of GNN are needed. For instance, this can happen when GNN is employed to pass the information from the element with boundary condition information (external force or fixture, e.g., element marked by "element with force information" in Fig. 1) to the element whose stress field is required to be solved ("target element" in Fig. 1a). One of the shortest message propagation paths from the boundary element to the target element is marked with red shown in Fig. 1a, which requires several layers of GNN to pass the information. However, deeply stacking the GNN layers can suffer from over-smoothing problems which cause indistinguishable graph-vertex embedding and make GNN eventually not predict anything (Li et al., 2020; Yang et al., 2020). In general, this problem restricts most GNN models to around 3 layers with only short-range graph-vertex interactions and limited message passing distance.

Given the limitations of GNN, from Table 1, it can be seen that the GNN is effective on two types of physical field regression problems: the first type is the problems which input a small-scale graph data into GNN with generally no more than 100 graph vertices, and usually requires data pre-processing or model simplification (Alet et al., 2019; Wang et al., 2020); the other type can solve the large-scale-graph problem, however, has only short-range graph-vertex interactions. This means the graph distance for the propagation of boundary information is relatively short since the elements embedding boundary information are rela-

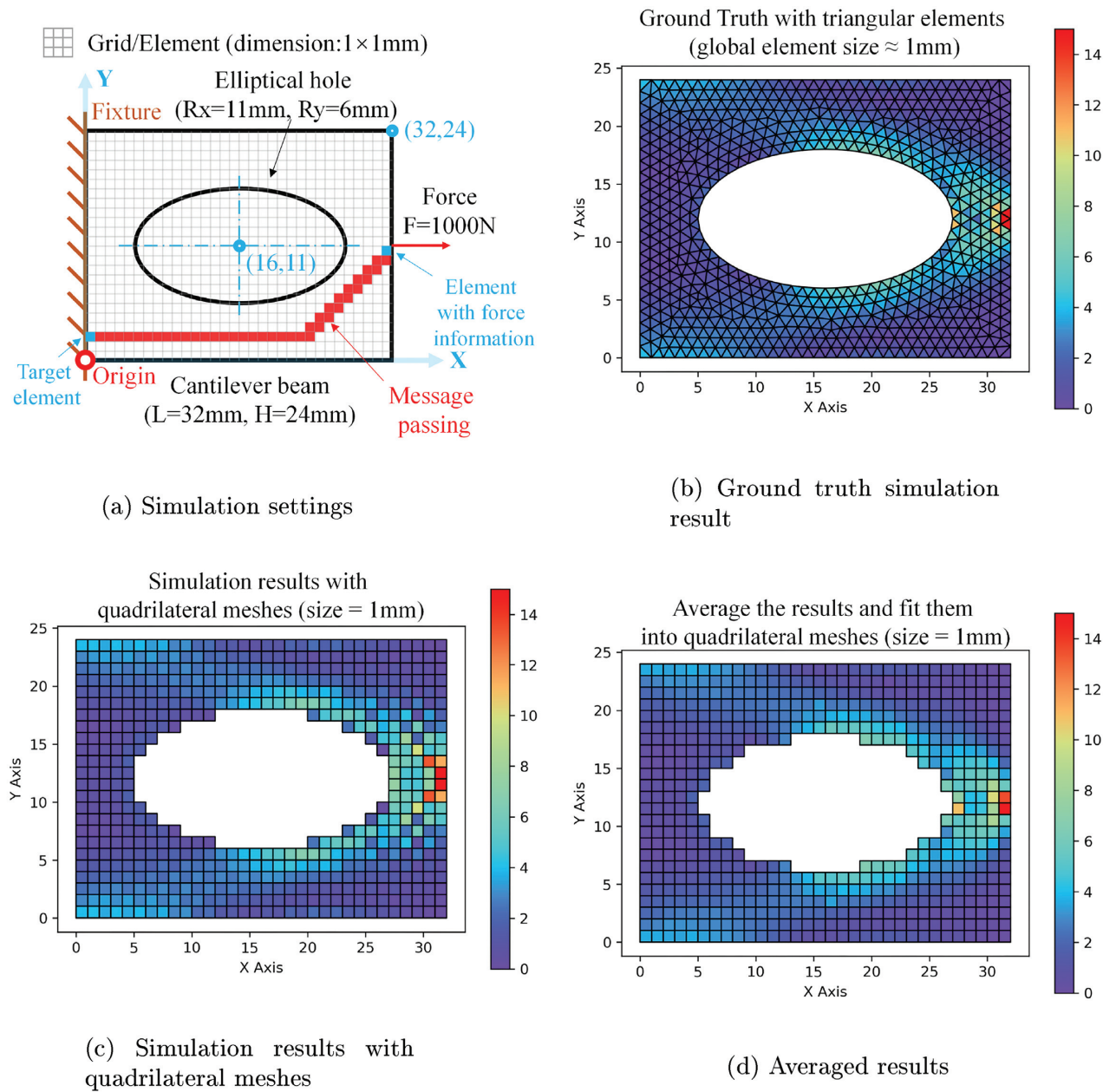


Figure 1: Comparison between different FEM simulation and prediction results.

Table 1: Researches for FEA surrogate models.

Model type	Graph size	Graph vertices interaction scale	Generalization	References
Conventional methods	–	–	Limited	Capuano and Rimoli (2019); Tamaddon-Jahromi et al. (2020)
CNN with quadrilateral or hexahedron meshes	–	–	Good	Nie et al. (2020, 2021); Koeppe et al. (2020); Khadilkar et al. (2019); Kutz (2017); Guo et al. (2016) Zhang et al. (2018a); Krokos et al. (2022); Courtecuisse et al. (2014)
GNN for small graph with triangular meshes	Small	Short range	Good	Alet et al. (2019); Wang et al. (2020)
GNN for large graph with triangular meshes	Large	Long range	Limited	Belbute-Peres et al. (2020); Guo and Buehler (2020)
Other GNN models	Large	Short range	Good	Ogoke et al. (2020); Pfaff et al. (2020); Sanchez-Gonzalez et al. (2020)

tively close to the target element whose physical field is required to be solved. For instance, Pfaff *et al.* (2020) developed an efficient GNN model for dynamic simulations based on triangular meshes. However, since the dataset contained sequential physical fields (displacement field, velocity field, stress field, etc.) and the input physical field for the current frame (or simulation step) was derived from the previous frame, the boundary condition information can slowly propagate from the boundary elements to the elements inside the model frame by frame. Then, the physical field of the target element was only determined by its surrounding elements that embed boundary information, which only requires the short-range graph-vertex interaction. The same happens when the regression model works on sequential physical field predictions (e.g., Sanchez-Gonzalez *et al.*, 2020). Ogoke *et al.* (2020) predicted drag force on a large graph using velocity fields, but only elements around the streamlined object contributed to the total force. Although some researchers successfully developed efficient GNN-based prediction models on large graphs, their applications were still limited. Belbute-Peres *et al.* (2020) developed a GNN-based fluid dynamics prediction model on large graphs by interpolating PDE solver's results, which had a relatively short-range graph distance for the propagation of boundary-condition information since the result from the PDE solver carried the model's boundary condition. Guo and Buehler (2020) developed a semi-supervised GNN model to predict optimized architected material structure on a large graph, but the selected training dataset was chosen from inside of the material which embedded boundary conditions. Also, the training needed to be conducted before each application in Guo and Buehler (2020). We have also tested some of the popular GNN techniques, e.g., GAT (Veličković *et al.*, 2017), UNet (Gao & Ji, 2019), DeepGCN (Li *et al.*, 2019, 2020), and DropEdge (Rong *et al.*, 2019) (details shown in Section 4) to solve this distant message passing problem but none of them can provide a good prediction result for FEA.

From the above, it can be seen that there is no ready-to-use generalized machine learning approach for solving unstructured-mesh-based boundary value problems, especially when long-range graph-vertex interactions are involved. FEA generally requires unstructured meshes to accurately approximate the boundary of the input model geometry. It usually provides finer meshes to the model's delicate structures to improve the computation accuracy and employs coarse meshes to save computational resources. The CNN-based FEA surrogate models have difficulty approximating the unstructured-mesh-based results, which becomes a bottleneck for the efficiency of related algorithms. The GNN-based approach shows its unique advantage of regressing unstructured-mesh-based results, however, a precise FEA model usually requires fine meshing to provide a high-resolution result, which can largely increase the graph size and therefore extends the propagation distance of the boundary information. This requires a deep GNN model and usually can cause the oversmoothing problem that is difficult to solve. Also, when GNN is applied to solve the static boundary value problems (different from the dynamic problems using the sequential physical field as its input with multiple frames (Pfaff *et al.*, 2020; Sanchez-Gonzalez *et al.*, 2020), the target element needs to gather all the boundary condition information in one frame (or simulation step) and then provides the prediction result. For instance, in the cantilever beam problem, the target element stress field is determined by all the elements with the boundary information (fixture, model's geometrical edges, and external forces, shown in Fig. 1), and this cannot be solved by any of the current approaches. Therefore,

in order to solve these problems and provide a potential general DL-based surrogate regression model, we focus on the most fundamental solid mechanics problem – the cantilever beam problem and develop a boundary oriented graph embedding (BOGE) approach. With the novel BOGE method, the GNN model works effectively on high-resolution triangular-mesh-based FEA simulation and predicts the accurate distribution of von Miss stress. Following the convention [23]–[26], we use topological optimization as an example to validate the abstract decision-making ability of the GNN with BOGE and it works efficiently. In this paper, Section 2 introduces our novel data representation which provides both multi-hop local adjacent information and shortcuts for global boundary information. Section 3 describes the details of our dataset preparation and the parametric designs for the FEA simulation results. Section 4 shows the model's training results that achieves the regression with mean square error (MSE) of 0.011706 [2.41% mean absolute percentage error (MAPE)] for stress field prediction and 0.002735 MSE (with 1.58% elements having error larger than 0.01) for topological optimization. The outperformance of our model's prediction accuracy as well as its irreplaceable properties on regression of FEA results shows our models' un-substitutable significance on fast FEA simulations. We summarize the advantages of our GNN based surrogate model as follows:

- (i) We propose a new BOGE approach for regression of physical fields and optimized designs in cantilever beam problems based on unstructured meshes (especially those based on triangular meshes) resulting from boundary value problems. The BOGE approach not only provides shortcuts for neighborhood meshes to smooth prediction results but also offers shortcuts of global boundary conditions to each internal element, which circumvents the message passing limitation of the MPNN framework on a large graph. Also, the BOGE largely compresses the graph size derived from the simulation model, enabling one graph vertex embed both mesh properties and geometrical edge properties, which leads to more efficient message propagation in GNN.
- (ii) We employ the parametric design method to prepare and standardize the synthesized dataset for cantilever beam simulation. The parametric dataset can be expanded with more design models to enhance the generalization capacity of our model. With the same design approach, the dataset for other types of physical-field simulators (e.g., temperature, velocity, etc.) and design optimizers can be developed.
- (iii) The BOGE is compatible with most GNN-based MPNN frameworks. With the simple combination of BOGE and other state-of-the-art GNN models, our GNN-based surrogate model shows its outstanding performance in approximating both physical fields and highly abstract optimization results. The model has the potential to be applied to most physical-field fitting problems with any type of polygon meshes or mixed types of meshes. The GNN-based surrogate approach paves the way for further DL-based physical field simulators and will benefit all computational design and simulation fields.

2. Concept of BOGE approach

The physical field distribution is generally derived by resolving partial differential equations under the given boundary condi-

tion. Modern computational design processes usually refer to this physical field result to evaluate and improve the design quality. For the stress analysis of the cantilever beam problem, denote the cantilever beam's geometry as Ω . Then, the global boundary condition $\partial\Omega$ of the problem can be expressed by

$$\partial\Omega = S_u + S_p \quad (4)$$

where S_u and S_p , respectively, represent the displacement (essential) and force/stress (natural) boundary condition. In an FEA model, the algorithm usually employs discretized approach to decompose the input geometry Ω into finite elements so that the complex boundary value problem can be simplified into solving the partial differential equation for each element. For the linear elasticity problem on isotropic materials, the canonical form of the relationship between mesh nodal displacement $\mathbf{U}$ and the global boundary condition $\mathbf{F}$ can be expressed by

$$\mathbf{KU} = \mathbf{F} \quad (5)$$

where $\mathbf{K}$ is the global stiffness matrix. Here, the global stiffness matrix $\mathbf{K}$ is assembled by the elemental stiffness matrices $\mathbf{k}_e$, which can be represented by an integration function of the element area A :

$$\mathbf{k}_e = \int_A \mathbf{B}_e^T \mathbf{C}_e \mathbf{B}_e dA \quad (6)$$

where $\mathbf{B}_e$ is the displacement differentiation matrix obtained by differentiation of displacements expressed through shape functions and nodal displacements; and $\mathbf{C}_e$ is the elasticity matrix constructed by Young's modulus (E) and Poisson's ratio (ν). Then, the nodal displacement field $\mathbf{U}$ can be solved. Based on $\mathbf{U}$, the stress field $\boldsymbol{\sigma} = [\sigma_x, \sigma_y, \tau_{xy}, \tau_{yx}]^T$ can be expressed by

$$\boldsymbol{\sigma} = \mathbf{C}_g \mathbf{B}_u \mathbf{U} \quad (7)$$

where $\mathbf{B}$ is the assembly of $\mathbf{B}_e$ matrices; and the block-diagonal matrix $\mathbf{C}_g$ is constructed from elemental elasticity tensor $\mathbf{C}$ that can be derived by $\mathbf{C}_e$ (Nie et al., 2020). From the equation, it can be seen that the element nodal stress state and the nodal displacement are not only determined by the global boundary condition $\partial\Omega$ but also influenced by local boundaries determined by its neighbor elements due to the assembly of $\mathbf{K}$. Specifically, if a GNN model is applied to solve the cantilever beam problem, the target element requires both long-range global boundary condition information and the short-range neighbor elements' properties to generate its stress field. Also, in a 2D cantilever-beam FEA simulation, three key items need to be fully defined to determine the final simulation solution: global boundary condition ($\partial\Omega$), pre-defined elements for the model geometry (Ω), and material properties. This section mainly introduces the BOGE approach which can comprehensively feed that information from the FEA model to the GNN. Section 2.1 introduces the features that a graph vertex needed to be embedded in BOGE. Section 2.2 introduces the graph links required by the BOGE approach.

2.1. Graph vertex embedding method

Most GNN-based FEA surrogate models embed mesh nodes as graph vertices for physical field simulation (Belbute-Peres et al., 2020; Guo & Buehler, 2020; Ogoke et al., 2020; Pfaff et al., 2020; Sanchez-Gonzalez et al., 2020). However, boundary conditions can be cast on mesh edges or mesh bodies, which is not convenient to be represented using element nodes. Also, it is natural to encode the material property inside the mesh body rather than mesh nodes, especially for the conditions when material property changes spatially, e.g., composite materials and non-homogeneous materials. The mesh nodes are between the mesh

bodies, which is hard to embed the change in mesh body properties. In order to solve this problem, according to the convention from the previous research (Pfaff et al., 2020), the mesh edge, as well as the mesh body and the mesh node, are considered graph vertices. However, compared with the condition when only element nodes are considered, embedding each mesh node, mesh edge, and mesh body as separate graph vertices will expand the graph size 3 times for triangular-mesh-based simulations, which can run out of computational resources or lead to an extremely long message passing distance for the propagation of boundary conditions. Therefore, in BOGE approach, we combine the properties of the mesh node, mesh edge, and mesh body into one graph vertex and embed the overall mesh features into the graph vertex to provide a compressed graph representation.

The format of our BOGE embedding features f_v is illustrated in Fig. 2. The first several channels encode the center of the mesh which provides the approximate position information of the mesh body. Material properties follow those channels which provide mesh body properties. After that, we encode the geometrical properties of mesh edges using the edge's normal vector and its distance to the mesh center to fully define the edge's geometrical position. Each edge's geometrical property follows the edge's physical property that encodes the global boundary condition of the simulation model. Following the edge properties, additional point-based information can be added to the data representation, e.g., the external force cast on the mesh.

In Fig. 2, we illustrate one triangular mesh encoding example using BOGE approach for solving the cantilever beam problem (details of the problem definition are explained in Section 3). For a 2D cantilever beam problem based on triangular-mesh simulation, the first two channels encode the mesh center (x_e, y_e) , which is calculated by the average of the coordinates for all the mesh nodes. The following mesh body property for a stress analysis simulation can be Young's modulus (E) and Poisson's ratio (ν) of the mesh material. After that, three mesh edges with their distances from the center (d_k) and their normal vectors $((x_{nk}, y_{nk}))$ are encoded (where $k = 1, 2$, and 3 representing each edge of the mesh). Following each edge geometry, the boundary condition is encoded by its boundary state (S_{uk}). We set the boundary state as 1 when the edge is on the outer contour of the input model geometry, while $S_{uk} = 2$ for edges on fixtures. After that, the data representation encodes the external 2D force (S_p) by attaching its relative position $((x_{F_{ext}}, y_{F_{ext}}))$, calculated according to the mesh center and its magnitude $((F_{ext-x}, F_{ext-y}))$ to the graph vertex features. Meshes without an external force need to be padded with zero to meet the requirement of graph embedding. Then, all the necessary information from a triangular mesh has been embedded into the graph vertex.

2.2. Adjacency information

The GNN requires the input of graph links E to figure out the message passing direction, which is encoded by the adjacency matrix. Conventionally, the mesh-based graph embedding is bounded by the geometry of the input mesh model (Ω), which only considers the graph links between the mesh and its neighboring meshes. However, in order to shorten the message passing distance to use shallow layers of GNN to regress the physical fields, in BOGE approach, we provide shortcuts for both the global boundary condition and the local neighboring meshes. Figure 3 illustrates both the neighboring graph links and the global shortcuts. Figure 3a shows the graph links between the target mesh (inside the mate-

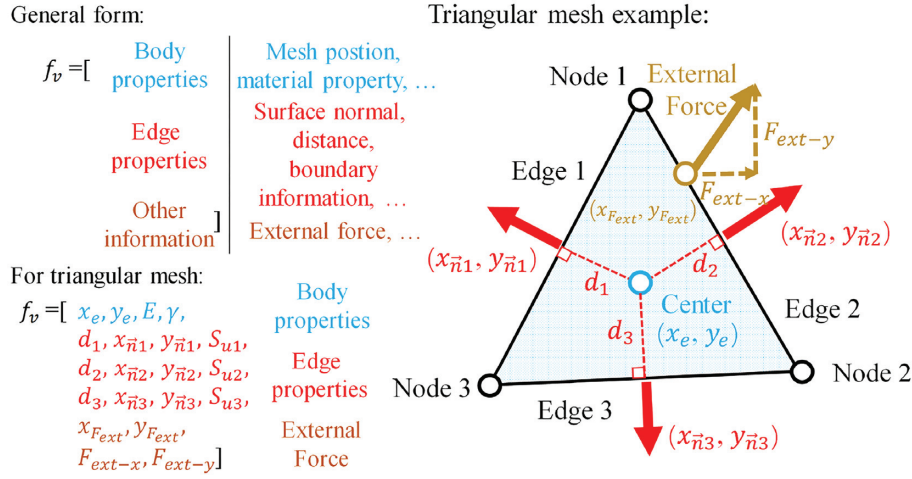


Figure 2: Encoding approach for triangular mesh.

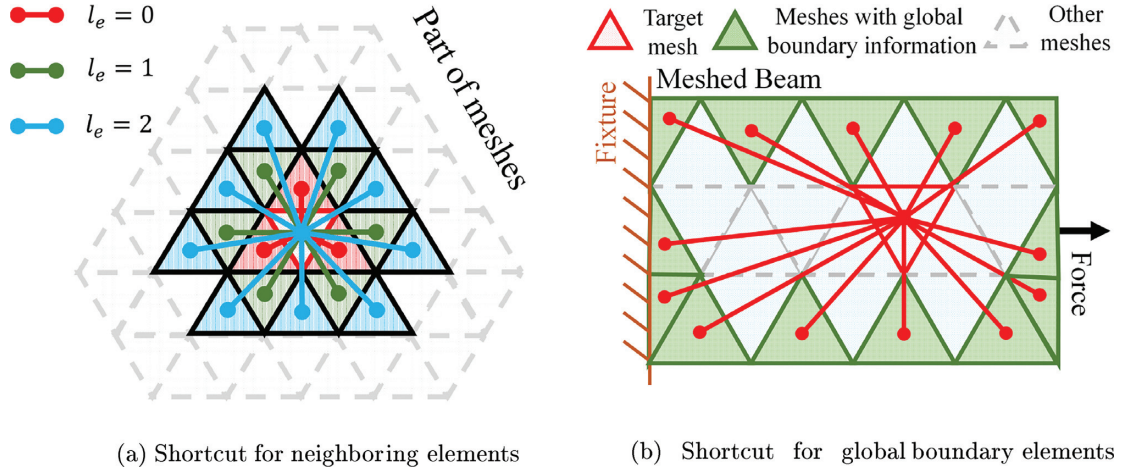


Figure 3: Encoding for graph connectivity and shortcuts.

rial) and its neighboring meshes while Fig. 3b presents the shortcut graph links between the target mesh and all the meshes with the boundary conditions.

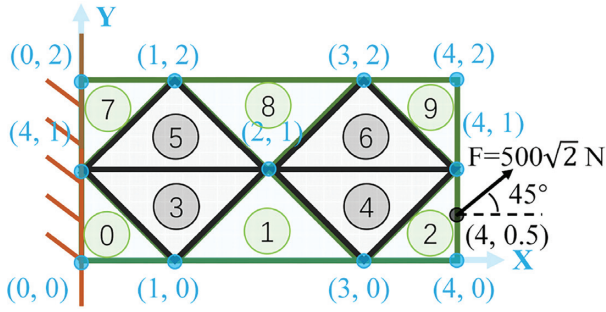
Most researchers focus on one-hop graph vertices, which are the meshes straightly connecting to the target mesh. Denote the distance between the target element and its neighbor element on the graph as l_e , then we represent the graph links between one-hop elements and the target element as $l_e = 0$, shown in Fig. 3a with red graph links. However, in order to provide more local information to the target mesh and smooth the physical-field results, we also consider two-hop ($l_e = 1$) and three-hop ($l_e = 2$) graph links (marked with green and blue graph links, respectively, in Fig. 3a) and encode those to the adjacency matrix. Adding information about meshes' local connectivity assists to provide more details of local boundary conditions, whose performance is presented in Section 4.

Then, we provide mesh connectivity between the target mesh and all the meshes that have the information of global boundary conditions ($\partial\Omega$) in order to provide the shortcut for solving the large-scale graph problem. In the cantilever beam problem, all the meshes with information about the model's geometrical edges, fixtures, and external forces, are connected to every internal mesh inside the beam, which is illustrated in Fig. 3b. With these graph-linking shortcuts, the GNN can assist any mesh inside

the simulation model to collect complete information from the boundary condition and model geometry, other than only passing the local mesh information to its neighborhood meshes with short-range distance. Therefore, the shallow-layer GNN structure can still regress high-resolution unstructured-mesh-based simulation results. Currently, there is no edge feature or edge weight considered, and the adjacency matrix is filled with "0" and "1". Future work can investigate the influence of different edge features, but the current encoding method is sufficient to solve the cantilever beam problem. The performance of this improvement is shown in detail in Section 4.

One example of BOGE representation for a rectangular beam with 10 triangular meshes is shown in Fig. 4. The adjacency matrix (E) in Fig. 4b uses one-hop graph links ($l_e = 0$). Some node features (e.g., Young's modulus and external forces) from Fig. 4c can be normalized before its input to the GNN. Since an encoding dense layer is usually applied at the first layer of the GNN, this encoding approach for f_v is only an example. Any node embedding method that can clearly input the mesh geometry and boundary condition to f_v can be applied in BOGE approach.

In this section, we present the BOGE approach to encoding triangular-mesh-based FEA problems in the graph structure. This concept can be generalized to other polygon-mesh simulations with more mesh-edge information. Padding the chan-



(a) Example beam with 10 triangular meshes

$$E = \begin{matrix} & \text{Mesh (graph node) \#} \\ & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{matrix} \\ \begin{matrix} \text{Mesh (graph node) \#} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

(b) Adjacency matrix (E) with $l_e = 0$

$$f_v = \begin{matrix} & x_e & y_e & E & \gamma & d_1 & x_{\bar{n}1} & y_{\bar{n}1} & S_{u1} & d_2 & x_{\bar{n}2} & y_{\bar{n}2} & S_{u2} & d_3 & x_{\bar{n}3} & y_{\bar{n}3} & S_{u3} & x_{F_{ext}} & y_{F_{ext}} & F_{ext-x} & F_{ext-y} \\ \begin{matrix} f_v \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \\ 6 \\ 7 \\ 8 \\ 9 \end{matrix} & \begin{bmatrix} 0.5 & 0.5 & 200 & 0.32 & 0 & 1 & 1 & 0 & 0.5 & 0 & -1 & 1 & 0.5 & -1 & 0 & 2 & 0 & 0 & 0 & 0 \\ 2 & 0.5 & 200 & 0.32 & 0.354 & 1 & 1 & 0 & 0.5 & 0 & -1 & 1 & 0.354 & -1 & 1 & 0 & 0 & 0 & 0 \\ 3.5 & 0.5 & 200 & 0.32 & 0.5 & 1 & 0 & 1 & 0.5 & 0 & -1 & 1 & 0 & -1 & 1 & 0 & 0.5 & 0 & 500 & 500 \\ 1 & 0.5 & 200 & 0.32 & 0.5 & 0 & 1 & 0 & 0.354 & 1 & -1 & 0 & 0.354 & -1 & -1 & 0 & 0 & 0 & 0 \\ 3 & 0.5 & 200 & 0.32 & 0.5 & 0 & 1 & 0 & 0.354 & 1 & -1 & 0 & 0.354 & -1 & -1 & 0 & 0 & 0 & 0 \\ 1 & 1.5 & 200 & 0.32 & 0.354 & 1 & 1 & 0 & 0.5 & 0 & -1 & 0 & 0.354 & -1 & 1 & 0 & 0 & 0 & 0 \\ 3 & 1.5 & 200 & 0.32 & 0.354 & 1 & 1 & 0 & 0.5 & 0 & -1 & 0 & 0.354 & -1 & 1 & 0 & 0 & 0 & 0 \\ 0.5 & 1.5 & 200 & 0.32 & 0.5 & 0 & 1 & 1 & 0 & 1 & -1 & 0 & 0.5 & -1 & 0 & 2 & 0 & 0 & 0 \\ 2 & 1.5 & 200 & 0.32 & 0.5 & 0 & 1 & 1 & 0.354 & 1 & -1 & 0 & 0.354 & -1 & -1 & 0 & 0 & 0 & 0 \\ 3.5 & 1.5 & 200 & 0.32 & 0.5 & 0 & 1 & 1 & 0.5 & 1 & 0 & 1 & 0 & -1 & -1 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

(c) Graph node features (f_v), details of the notation are shown in Fig. 2

Figure 4: Example for BOGE encoding.

nels with zeros to keep the uniform graph-vertex feature size for all the meshes, data representation can encode simulation with mixed types of unstructured mesh (e.g., triangular meshes and quadrilateral meshes) and will incur only a small amount of extra computational resources since GNN generally employs sparse tensor production during training and computing. Our BOGE approach largely compresses the graph size and provides shortcuts for the boundary conditions, which can improve the long-range graph-vertex interactions in boundary value problems. It is compatible with most MPNN-based GNN structures and has the potential to be generalized to solve other boundary value problems. This can also assist in mesh-based design optimization.

3. Dataset Preparation

In order to validate the efficiency of this approach, we focus on two aspects of the cantilever beam problem: stress field prediction and topology optimization. We summarized all the cantilever beam models provided in Nie et al. (2020) and created a parameterized synthetic FEA simulation dataset. According to Zhang et al. (2018b), Peddireddy et al. (2021), and Fu et al. (2021), we provide uniform random values to provide design parameters for cantilever beams to improve the repeatability and portability of the dataset. Details on dataset preparation are explained in this section.

3.1. Stress field simulation with the force boundary condition

According to Nie et al. (2020), a balanced FEA simulation dataset with nine types of cantilever beam models is summarized and shown in Fig. 5 which includes all the beam geometries from Nie et al. (2020). All the design parameters for the cantilever beams geometry are generated uniformly randomly according to the range presented in Fig. 5 with a minimum increment of 0.1 mm. In the case of beam structures with holes, certain constraints have been considered to prevent thin walls with a thickness of <1 mm. The fixtures ($S_u = 0$) in this dataset are always on the left edge of the cantilever beam while the external force (S_p) is exerted on a random position selected from the right edge of the beam. The magnitude and the angle of the force are uniform-randomly generated with a minimum increment of 100 N and $\frac{\pi}{6}$, respectively, ranging from 100 to 1000 N and from 0 to 2π . Although the generated cantilevered structures cannot represent all the beam structures, the selected ones represent the most common structures in the mechanical areas (Nie et al., 2020). Also, following a similar parametric design pattern, other features can be considered and added to the dataset in the future according to previous synthetic CAD design research (Fu et al., 2021; Peddireddy et al., 2021; Zhang et al., 2018b).

The ground truth of the stress field distribution is calculated by a commercial FEA software – ABAQUS (Abaqus, 2011). The

#	Beam Shape	Parameters	#	Beam Shape	Parameters
1		$H \in [12, 24]$ $L \in [16, 32]$ $h \in [0, 32 - H]$ $y_F \in [0, H]$	2		$H \in [12, 24]$ $L \in [16, 32]$ $h \in [0, 32 - H]$ $R_x \in [2t, \text{int}(\frac{L}{4})]$ $R_y \in [2t, \text{int}(\frac{H}{4})]$ $x_e \in [R_x + t, L - R_x - t]$ $y_e \in [R_y + h + t, h + L - R_y - t]$ $y_F \in [0, H]$
3		$H \in [12, 24]$ $L \in [16, 32]$ $h \in [0, 32 - H]$ $R_x \in [\text{int}(\frac{L}{4}), \text{int}(\frac{L}{2})]$ $R_y \in [2t, \text{int}(\frac{H}{4})]$ $x_e \in [\text{int}(\frac{L}{4}) - R_x, \text{int}(\frac{L}{2}) - R_x]$ $y_e \in [R_y + h + t, h + L - R_y - t]$ $y_F \in [0, H]$	5		$H \in [12, 24]$ $L \in [16, 32]$ $W \in [6, H - 2t]$ $h \in [0, 32 - H]$ $w \in [0, H - W]$ $R_x \in [2t, \frac{L-2t}{2}]$ $R_y \in [2t, \frac{H-2t}{2}]$ $x_e \in [R_x + t, L - R_x - t]$ $y_e \in [R_y + h + t, h + L - R_y - t]$ $y_F \in [0, H]$
4		$H \in [12, 24]$ $L \in [16, 32]$ $W \in [6, H - 2t]$ $h \in [0, 32 - H]$ $w \in [0, H - W]$ $y_F \in [0, H]$	6		$H \in [12, 24]$ $L \in [16, 32]$ $W \in [6, H - 2t]$ $h \in [0, 32 - H]$ $w \in [0, H - W]$ $R_x \in [\frac{L}{2}, L - t]$ $R_y \in [2t, \frac{H-2t}{2}]$ $x_e \in [R_x + t, L - R_x - t]$ $y_e \in [R_y + h + t, h + L - R_y - t]$ $x_F = L$ $y_F \in [0, H]$
7		Same to 4; Centers for the curve edges are on the Y axis.	8		Same to 5; Centers for the curve edges are on the Y axis.
9		Same to 6; Centers for the curve edges are on the Y axis.			

Figure 5: Dataset design parameters.

chosen material for the simulations is A36 steel with Young's modulus of 200 GPa and a Poisson's ratio of 0.32. The process of triangular tessellation for the cantilever beam structures is carried out by ABAQUS built-in functions with an approximate global mesh size of 1.0 mm. The settings for the deviation fac-

tor and minimum size factor are both chosen to be the default value of 0.1 mm to guarantee a fine triangular-mesh approximation. ABAQUS conducts the FEA algorithm explained in Section 2 to calculate the displacement field and the stress field for the cantilever beams. We choose von Mises stress (σ_{vm}) as

Table 2: Details for the dataset.

Dataset	Number of data in training/ validating/testing subsets	Physical quantity	Physical quantity Mean std dev	(Min.,max.)	Avg. number of meshes
Stress prediction (45k)	31.5k/6.75k/6.75k	von Mises stress (σ_{vm}) – MPa	2.0642.239	(0,74.09)	857
Stress prediction (180k)	126k/27k/27k	von Mises stress (σ_{vm}) – MPa	2.0572.232	(0,74.09)	857
Topology optimization (45k)	31.5k/6.75k/6.75k	Volume density (z_e)	0.2980.389	(0,1)	857
Stress prediction – displacement BC (45k)	31.5k/6.75k/6.75k	von Mises stress (σ_{vm}) – MPa	1.6561.980	(0,79.81)	857

(a) Details for the data property in dataset

Dataset	Execution time (s)	Input processing (s)	Job time (s)	Number of equations	Baseline (s)
Stress prediction (45k)	11.53	1.201	0.947	946	0.947
Stress prediction (180k)	11.58	1.201	0.947	946	0.947
Topology optimization (45k)	489.09	38.584	23.321	946	61.905
Stress prediction – displacement BC (45k)	11.55	1.204	0.940	947	0.940

(b) Details for the dataset generation time

the target stress field for the final regression, which can be represented by

$$\sigma_{vm} = \sqrt{\sigma_x^2 + \sigma_y^2 - \sigma_x \sigma_y + 3 \tau_{xy}^2}. \quad (8)$$

We encode the triangular meshes with their boundary condition into the graph according to the BOGE approach explained in Section 2. The external force has been normalized in vertex features and the material properties have not been included since only one material is considered. Table 2 provides details regarding the generated dataset.

3.2. Stress field simulation with the displacement boundary condition

Section 3.1 only considers the condition when the point load force is applied on the end of the cantilever beam. However, in order to investigate the performance of our method in different boundary conditions, we also develop a new dataset (“Stress prediction – displacement BC (45k)” in Table 2) to consider the effect when the displacement boundary condition is applied to the right side of the cantilever beam as the input.

We follow the same data generation strategy as we have explained in Section 3.1 except we substitute the point load force with a point displacement. The displacement ranges from 0.1 to 1 μm with a minimum increment of 0.1 μm , which can generate stress fields with the same magnitude as those in the dataset generated by the point load mentioned in Section 3.1, shown in Table 2. This displacement boundary condition has also been normalized in input vertex features. All other parameters keep the same. For the shape of the beam, we also follow Fig. 5 to create this new balanced dataset but only substitute the “force” in the figure with the displacement.

3.3. Topology optimization

Based on the simulated stress field, we calculate topological optimization results to validate the abstract decision-making capability of our approach. ABAQUS employs the density-based simple isotropic material with penalization (SIMP) method to optimize the cantilever beam structure. The optimization process works toward minimizing the compliance function $C(z)$ which is the sum of the strain energy of all the

elements:

$$\left. \begin{array}{l} x_{\min} : C(z) = \mathbf{U}^T \mathbf{K} \mathbf{U} = \sum_{e=1}^N (z_e)^p \mathbf{u}_e^T \mathbf{k}_e \mathbf{u}_e \\ \text{s.t.} \quad \frac{V(z)}{V_0} = V_f \\ \mathbf{K} \mathbf{U} = \mathbf{F} \\ 0 < z_{\min} \end{array} \right\} \quad (9)$$

where $V(z)$ and V_0 represent the optimized material volume and the model’s design domain volume; and V_f is the volume fraction, which has been chosen as 0.3 for the final optimization target. Variable z represents the element material density that varies from $z_{\min}$ (void if 0, but generally non-zero to avoid singularity) to 1 (the full solid); and p is the penalization power whose value is usually 3 (Lee et al., 2020; Nie et al., 2021). Using ABAQUS built-in optimization algorithm, we set the maximum design cycle as 25 and solve the SIMP problem. The optimized cantilever beams have a final $V_f \in (0.29, 0.3]$ which satisfies the required design target. The optimized z_e for each element has been exported as the regression ground truth for GNN to be trained.

3.4. Dataset summary

We finish all the dataset preparation tasks on a personal computer with an Intel i7-10700k CPU. The data generation algorithm runs in less than 10 threads with a total CPU occupation of less than 90%, which has not influenced the evaluation of the computation efficiency of ABAQUS. The average computation time is shown in Table 3b. It should be noticed that the ABAQUS simulation includes input checking, input processing, job-solving, and other unknown tasks. The “Execution time” from Table 3b counts the total execution of the job from the time when the “.inp” file (ABAQUS input file) is submitted to the running ABAQUS on a command shell to the time when the entire job is finished. Besides this, the “Input processing” time only includes the time for data loading and data preprocessing while the “Job time” only counts the time taken on solving equation (4). In this paper, we refer to other DL-based cantilever-beam-problem regressors (Lee et al., 2020; Nie et al., 2020, 2021; Zhang et al., 2019)) and use the “Job time” as our baseline to validate the performance of our model. The average computation time for the stress prediction is around 947 ms. For the topological optimization, since there are several design cycles and it requires input work before each iteration, we also count the “Input processing” time for the total computation time, which is around 62 s. We provide three balanced datasets with 45k sim-

Table 3: Stress prediction results with displacement boundary condition (notations for neural network layers are explained in Table 6; training/validating/testing accuracy are presented without regularization).

No.	Models	Neural network structure	Dataset	MSE : training / validating / testing	MAPE : training/ validating / testing
#1	DeepGCN(3 layer)	FC(18,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	Stress prediction – Displacement BC (45k)	1.2893×10^{-2} / 1.3436×10^{-2} / 1.3486×10^{-2}	2.06% / 2.06% / 2.07%

(a) Model structure and prediction accuracy

No.	Models	Dataset	Trainable paramteres	Avg. training / validating / testing time per epoch (s)	Avg. single file loading time (ms)	Avg. single file testing time (ms)
#1	DeepGCN(3 layer)	Stress prediction – Displacement BC (45k)	52164	909.974/103.787/ 104.739	12.042	11.189

(b) Details for the model information**Table 4:** Stress prediction results (notations for neural network layers are explained in Table 6; training/validating/testing accuracy are presented without regularization).

No.	Models	Neural network structure	Dataset	MSE : training / validating / testing	MAPE : training / validating / testing
#1	DeepGCN(3 layers) with BOGE ($l_e = 0$)	FC(18,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	Stress prediction (45k) with BOGE ($l_e = 0$)	1.6383×10^{-2} / 1.7319×10^{-2} / 1.7127×10^{-2}	2.83% / 2.87% / 2.88%
#2	DeepGCN(3 layers) with BOGE ($l_e = 1$)	FC(18,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	Stress prediction (45k) with BOGE ($l_e = 1$)	2.1031×10^{-2} / 2.1804×10^{-2} / 2.1737×10^{-2}	3.02% / 3.04% / 3.08%
#3	DeepGCN(3 layers) with BOGE ($l_e = 2$)	FC(18,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	Stress prediction (45k) with BOGE ($l_e = 2$)	2.6551×10^{-2} / 2.7494×10^{-2} / 2.7302×10^{-2}	3.38% / 3.40% / 3.44%
#4	DeepGCN(3 layers) with BOGE ($l_e = 0$) (Best result)	FC(18,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	Stress prediction (180k) with BOGE($l_e = 0$)	1.1598×10^{-2} / 1.1658×10^{-2} / 1.1706×10^{-2}	2.43% / 2.43% / 2.41%

(a) Model structure and prediction accuracy

No.	Models	Dataset	Trainable paramteres	Avg. training / validating / testing time per epoch (s)	Avg. single file loading time (ms)	Avg. single file testing time (ms)
#1	DeepGCN(3 layers) with BOGE ($l_e = 0$)	Stress prediction (45k) with BOGE	52164	923.925/106.070/ 105.932	12.124	10.582
#2	DeepGCN(3 layers) with BOGE ($l_e = 1$)	Stress prediction (45k) with BOGE	52164	966.219/106.702/ 106.612	12.214	10.680
#3	DeepGCN(3 layers) with BOGE ($l_e = 2$)	Stress prediction (45k) with BOGE	52164	1013.272/110.427/ 110.560	12.531	11.183
#4	DeepGCN(3 layers) with BOGE ($l_e = 0$)	Stress prediction (180k) with BOGE	52164	3547.158/ 417.707 / 417.662	12.124	10.582

(b) Details for the model information

ulation results (5k simulations for each class from Table 3 with 3.5k/0.75k/0.75k simulations for training/validating/testing subsets) for stress field distribution predictions and topological optimizations.

Another balanced dataset with 180k simulation results (20k simulations for each class from Table 3 with 14k/3k/3k simulations for training/validating/testing subsets) for stress field predictions only is generated to investigate the influence of the dataset size.

The parameterized synthetic dataset can be generalized to other fields related to the FEA surrogate DL model. The synthetic dataset sometimes has an expensive time cost that can become

a bottleneck for our approach to be a general-purpose surrogate model. However, the dataset prepared by physical experiments can also serve the training task. The stress fields can be measured by installing strain gauges on real cantilever beams in the area whose position corresponds to that of the predefined meshes in the simulation. Multiple data can be generated within a short time when the external boundary conditions change. In this condition, the physical experiment can serve as a fast Partial Differential Equation (PDE) solver which assists to generate sufficient data for the surrogate model. All of these ideas contribute to the realization of a generalized model with high-precision predictions under various working conditions.

Table 5: Topology optimization prediction results (notations for neural network layers are explained in Table 6; training/validating/testing accuracy are presented without regularization).

No.	Models	Neural network structure	Dataset	MSE : training / validating / testing	MAPE : training/ validating / testing
#1	DeepGCN(3 layer) (Best result)	FC(18,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	Topology optimization (45k)	2.627×10^{-3} / 2.731×10^{-3} / 2.735×10^{-3}	22.26% / 22.65% / 22.53%
#2	DeepGCN(3 layer) with σ attached	FC(21,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	Topology optimization (45k) with σ attached	3.285×10^{-3} / 3.405×10^{-3} / 3.378×10^{-3}	21.35% / 21.91% / 21.57%

(a) Model structure and prediction accuracy

No.	Models	Dataset	Trainable paramteres	Avg. training / validating / testing time per epoch (s)	Avg. single file loading time (ms)	Avg. single file testing time (ms)
#1	DeepGCN(3 layer)	Topology optimization (45k)	52164	923.925/106.070/ 105.932	12.290	10.324
#2	DeepGCN(3 layer) with σ attached	Topology optimization (45k) with σ attached	52356	966.219/106.702/ 106.612	12.356	10.655

(b) Details for the model information

4. Neural Network Training and Results

In order to present the efficiency of the BOGE approach, we compare its performance with the conventional graph embedding methods (without any graph shortcuts) on different state-of-the-art GNN structures, e.g., GCN (Kipf & Welling, 2016), GAT (Veličković et al., 2017), UNet (Gao & Ji, 2019), and DeepGCN (Li et al., 2019, 2020). We choose the MSE function as the loss function and the evaluation metric to quantify our prediction performance (Koeppe et al., 2020; Nie et al., 2020). The predicted outputs – the magnitude of von Mises stress σ_{vm} and the volume density z_e , are all 1D tensors which can be written as $\hat{p} = (\hat{p}_1, \hat{p}_2, \dots, \hat{p}_n)$, while the ground truth results can be represented by $p = (p_1, p_2, \dots, p_n)$. The MSE can be represented as

$$MSE = \frac{1}{n} \sum_{i=1}^n (p_i - \hat{p}_i)^2. \quad (10)$$

According to Koeppe et al. (2020), we also employ the MAPE to present the error rate in percentage, which can be expressed by

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left(\left| \frac{p_i - \hat{p}_i}{p_i + \varepsilon} \right| \right) \quad (11)$$

where we set $\varepsilon = 0.01$ to avoid computation error when the ground truth result is zero or extremely small.

All the codes have been written in Pytorch (shared in Github; https://github.com/stxyfu/Stress_GNN) with Pytorch geometric library (Fey & Lenssen, 2019) and run on an NVIDIA RTX 3090 GPU and an Intel i7-10700k CPU. For all training experiments, the ADAM optimization algorithm has been employed to adjust the hyperparameters in the model. The training batch size is set to 8 to make maximal use of GPU memory and provide a precise training gradient while the learning rate is 0.01 obtained through the grid search to provide the best performance of the final GNN models. For the final selected GNN model (Tables 3, 4, and 5), we have tested the batch size of [1,2,4,8] and the learning rate of [0.1, 0.05, 0.01, 0.005, 0.001]. The final GNN model follows the structure of DeepGCN (Li et al., 2019, 2020) and we have tested [2,3,4,5] layers to improve the final prediction accuracy. For the number of hidden

channels in each layer of DeepGCN, we have tried [64,128,256,512]. We have also tested to revise the structure of the encoder, decoder, and the internal Multi-layer Perceptron (MLP) inside GEN (Li et al., 2020), and the presented model provides the best result. Dropout layers have not been applied due to their worse results according to our tests. Five hundred training epochs have been employed for each experiment to ensure the GNNs are all fully trained with suitable prediction accuracy. We present all the neural network structures in tables with their corresponding number of trainable parameters and training/validating/testing times per epoch. Since the training/validating/testing tasks also include the data loading, we have also included “Avg. single file loading time” to present the time consumed by loading the single data file from the hard drive to the GPU memory. The “Avg. single file testing time” from the following tables illustrates the forward propagation time when the data file has already been loaded to the GPU. Here, the testing time is only counted by the batch size of 1. Since the training batch size is 8, the real application can run with a much faster prediction. The entire training process for the selected GNN models (Tables 3, 4, and 5) takes around 125 h on 45k datasets and 493 h on the 150k dataset. The training time can be largely reduced if multiple high-performance GPUs are employed.

4.1. Stress field prediction with conventional graph embedding

We first investigate the regression capability of GNNs with conventional graph embedding methods on the 45k stress field prediction dataset, shown in Table 2. Conventional graph embedding methods only consider one-hop graph links (with $l_e = 0$ in our condition) whose adjacency matrix does not contain any shortcuts for boundary elements. Using this graph embedding method, shallow layer GNN (“GCN(3 layers)” in Table 6) cannot pass the boundary information to all the internal elements which cannot perform any regression at the end of the training. Figure 6c illustrates that, with the 3-layer GCN, the boundary information only propagates through limited meshes near the model’s geometrical boundary and cannot reach the internal elements inside the ma-

Table 6: Training results with different GNN structures and graph embedding methods (training/validating/testing accuracy are presented without regularization).

No.	Models	Neural network structure	Normalization	MSE : training / validating / testing	MAPE : training / validating / testing
#1	GCN(3 layers)	FC(18,64) → GCN(64,64)×3 → FC(64,1)	LayerNorm	0.48721 / 0.50387 / 0.50327	29.89 % / 29.95 % / 30.96%
#2	GCN(8 layers)	FC(18,256) → GCN(256,256)×8 → FC(256,128) → FC(128,1)	LayerNorm	0.63016 / 0.65133 / 0.64831	36.82% / 36.78% / 37.91%
#3	DeepGCN(3 layers)	FC(18,64) → DeepGCN(GEN(64,64,2))×3 → FC(64,1)	LayerNorm	0.47104 / 0.49264 / 0.49142	28.55% / 28.64% / 29.67%
#4	DeepGCN(8 layers)	FC(18,256) → DeepGCN(GEN(256,256,2))×8 → FC(256,128) → FC(128,1)	LayerNorm	0.40521 / 0.43995 / 0.44090	25.26% / 25.52% / 26.40%
#5	GAT	FC(18,256) → DeepGCN(GAT(256,256,2))×8 → FC(256,128) → FC(128,1)	LayerNorm	0.628749 / 0.649594 / 0.644869	35.30% / 35.41% / 36.42%
#6	gUnet	FC(18,32) → gUnet(64,1,4)	-	0.53886 / 0.56052 / 0.55579	33.09% / 33.07% / 34.36%

FC(f_{in}, f_{out}): Fully connected layer;GCN(f_{in}, f_{out}): Graph convolutional layer; Kipf and Welling (2016)GAT(f_{in}, f_{out}, H_{GAT}): Graph attentional layer; Veličković et al. (2017)gUnet($f_{in}, f_{out}, D_{gUnet}$): Graph U-net; Gao and Ji (2019)GEN(f_{in}, f_{out}, L_{MLP}): GENeralized Graph Convolution layer; Li et al. (2020)

DeepGCN(*): DeepGCN layer (Graph convolutional layer→Normalization→Activation→ResGCN→Dropout); Li et al. (2019, 2020)

 f_{in} : Number of input channels; f_{out} : Number of output channels; H_{GAT} : Number of multi-head-attentions; D_{gUnet} : The depth of the U-Net architecture; L_{MLP} : The number of MLP layers;**(a) Model structure and prediction accuracy**

No.	Models	Trainable paramteres	Avg. training / validating / testing time per epoch (s)	Avg. single file loading time (ms)	Avg. single file testing time (ms)
#1	GCN(3 layers)	16193	102.903/28.491/28.769	2.231	3.362
#2	GCN(8 layers)	568321	226.920/32.712/32.794	2.231	8.239
#3	DeepGCN(3 layers)	52164	217.103/29.012/29.060	2.231	3.151
#4	DeepGCN(8 layers)	2120713	743.895/46.471/46.573	2.231	9.436
#5	GAT	7910657	624.782/43.211/42.352	2.231	8.688
#6	gUnet	32194	447.420/69.672/69.888	2.231	52.691

(b) Details for the model information

terial. The deep-layer GNN can save the weighted features passed from other elements in its hidden state h according to equation (1) and provides a relatively long-range message passing. According to our statistics, the largest graph distance between two unstructured meshes is around $l_e = 250$ which requires at least 8 layers of GNN for general MPNN layers. However, deep GNN usually incurs oversmoothing that makes the embedded features similar and prevents the further regression process. For example, in Fig. 6e, the 8-layer GCN (“GCN(8 layers)” in Table 6) cannot predict anything but only outputs uniform values after 500 training epochs, which illustrates this extreme oversmoothing phenomenon. In order to solve this problem, researchers have employed different methods to mitigate the influence of oversmoothing and realized a few state-of-art GNN structures with deep GNN layers. From those methods, the most popular anti-oversmoothing techniques are DeepGCN (Li et al., 2019, 2020), attention layers (e.g., GAT; Veličković et al., 2017), and DropEdge (Rong et al., 2019). Some other GNN structures, though have been developed for other pur-

poses, can also reach deep layers with sufficient accuracy, e.g., the g-U-Net (Gao & Ji, 2019). However, most of these methods mainly work on classification problems. For the regression problem in our condition, extra validations are needed. Therefore, we have tested those models using the conventional graph embedding approach and shown those results in Table 6 and Fig. 6.

DeepGCN employs the generalized graph convolution layers (GEN; Li et al., 2020), layerNorm (Li et al., 2019), and ResNet structure (He et al., 2016), making GNN sufficiently robust against oversmoothing and can reach 56 layers with high classification accuracy. However, 8-layers DeepGCN (“DeepGCN(8 layers)” in Table 6) with the ResGCN backbone (Li et al., 2019) still cannot reach a high accuracy and performs some oversmoothing shown in Fig. 6i. GAT employs the attention mechanism that provides weighted attention coefficients to various graph features to ensure that GNN can pass the important information through deep layers. We employ the same DeepGCN backbone, but substitute GAT (with four attention heads) for GEN and try to im-

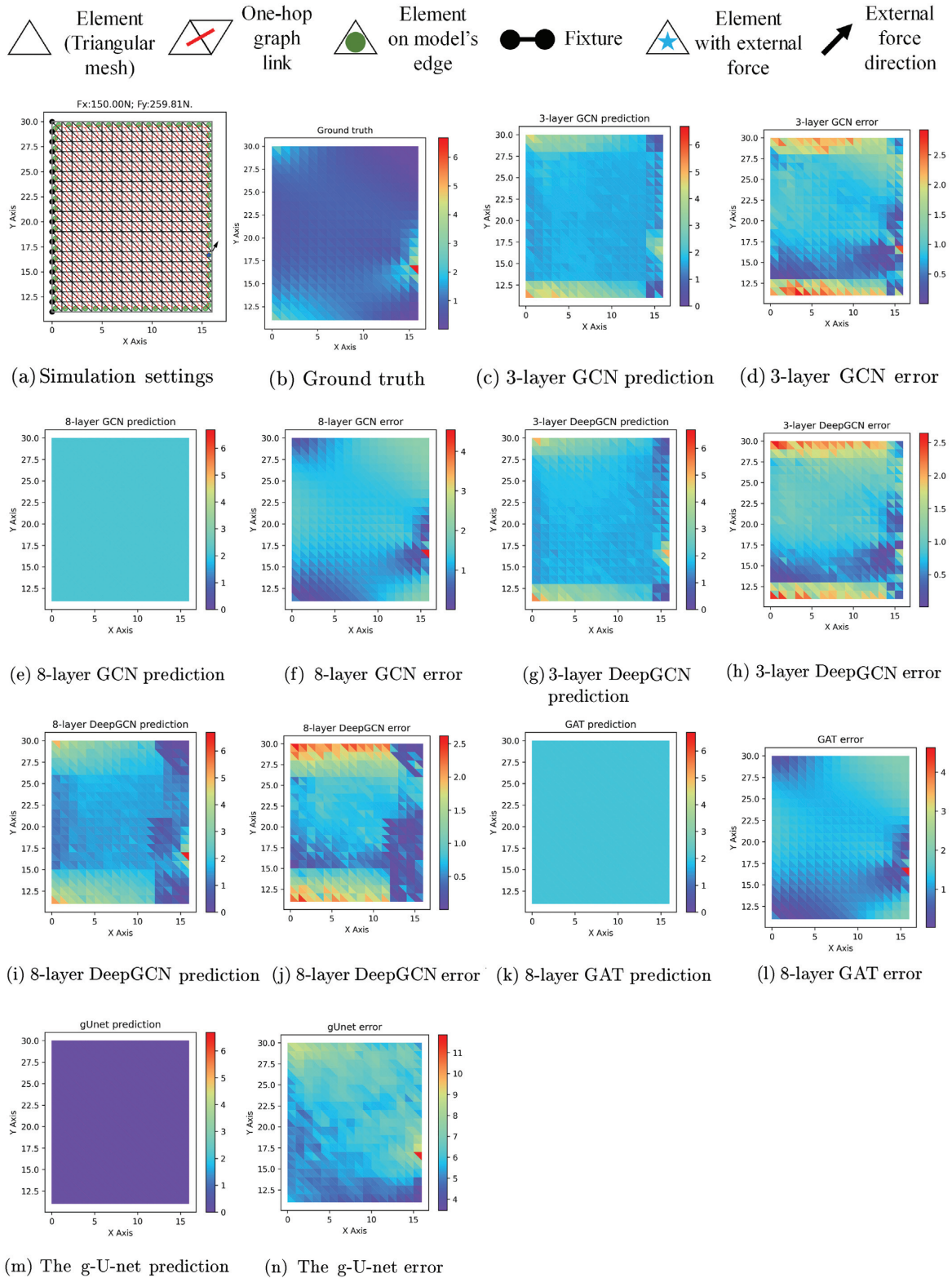


Figure 6: Prediction results for conventional graph embedding methods.

prove the prediction accuracy, shown as “GAT” in Table 6. According to our tests, the combination of GAT and the DeepGCN backbone performs better than simply stacking GAT layers on our dataset. However, the training result still appears to be incapable of obtaining acceptable results and shows an extremely

oversmoothed result, shown in Fig. 6k. The g-U-Net can encode and decode both high-level features and local features which can reach 5 layers in Gao and Ji (2019). However, each graph pooling layer drops both graph vertices (mesh elements) and graph links. When only one-hop graph links ($l_e = 0$) are considered,

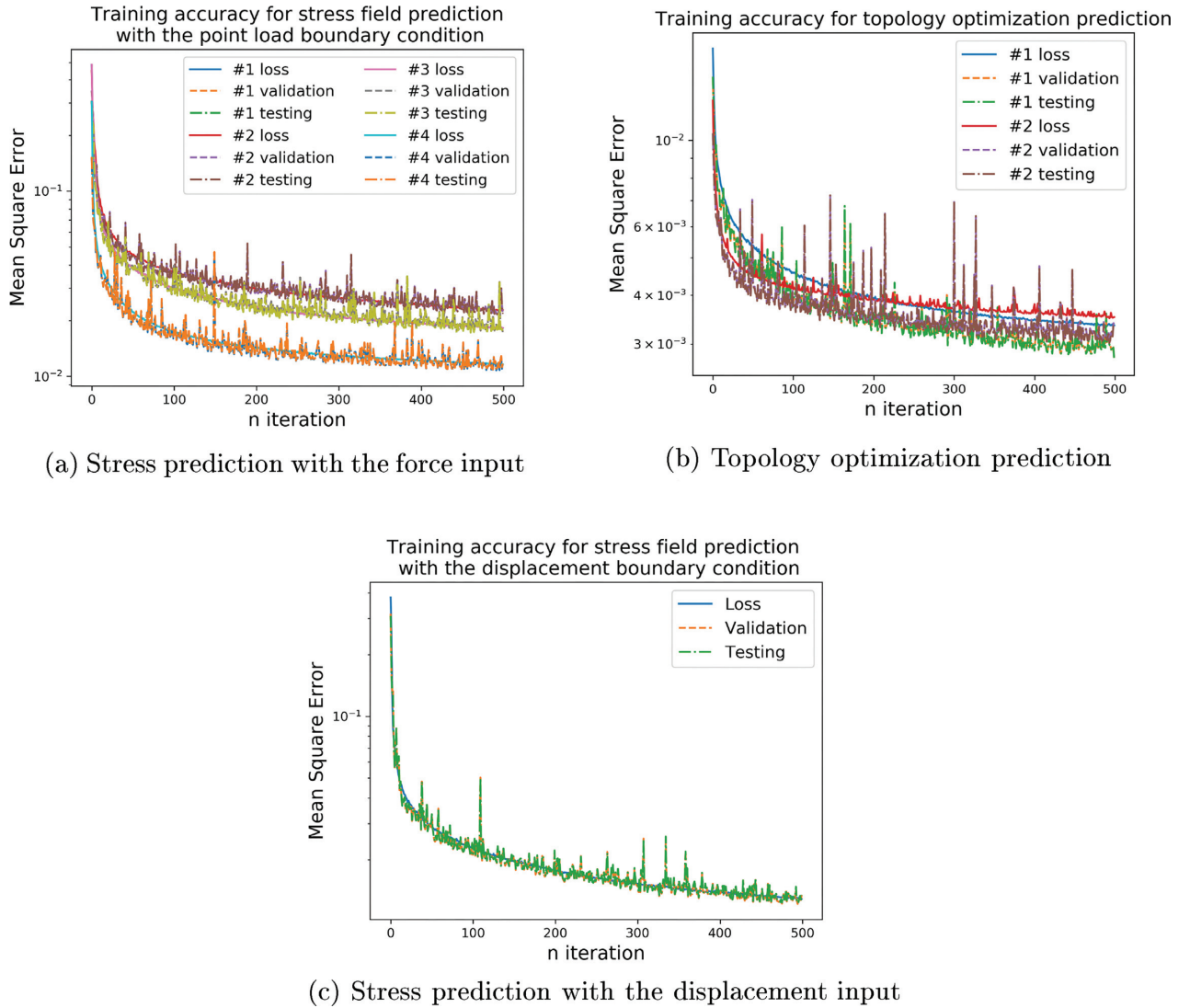


Figure 7: Training loss, validation accuracy, and testing accuracy.

with a very sparse adjacency matrix, stacking pooling layers in the g-U-Net makes the adjacent matrix too sparse so that only a few graph links remain in high-level feature scope, which cannot pass ample information to other elements through GNN and ruin the g-U-Net's prediction capability. Also, a deeper g-U-Net can still incur the extreme oversmoothing problem shown in Fig. 6m (with the training result shown as "gUnet" in Table 6). Other anti-oversmoothing techniques like DropEdge can ruin the integrity of the graph, which is not suitable for our problem as we have tested. The overall training results show that deep-layer GNNs with the conventional graph embedding approach cannot simply regress the boundary value problem with long-range graph-vertex interactions.

The tuning of the models in Table 6 follows the same grid search strategy as that for the final selected models to provide a good comparison. For shallow layer models (model #1 and #3 in Table 6), we select the number of hidden channels as 64 to compare them with our final model. For deep layer models (model #2, #4, and #5), we have tested the number of the hidden channels as [64, 128, 256, 512] and none of these models can make a good prediction. We present the deep layer model with 256 hidden channels in the table since a few 8-

layer model with 512 hidden features can exceed the maximum memory of the GPU. We refer to Gao and Ji (2019) and tested g-U-Net with a number of the hidden channels of [32, 64, 128] and the prediction result does not perform well. Overall, none of these models can provide a good prediction result as we have investigated.

4.2. Stress field prediction with BOGE for the force boundary condition

The BOGE approach shows its unique benefits for regressing the results of large-scale boundary value problems with simple shallow-layer GNNs. By providing shortcuts for both global boundary conditions and local information of neighboring elements, GNNs with BOGE bypass the message passing limitation of the MPNN framework, and all the internal elements can directly get the boundary condition information to solve the boundary value problem. We have employed the 3-layers DeepGCN and BOGE with $l_e = 1, 2$, and 3 graph links to validate its performance on the 45k stress field prediction dataset. The results are shown in Table 4.

Compared with the predicted results from "DeepGCN(3 layers)" in Table 6, BOGE with $l_e = 0$ graph links (model #1 in Table 4) largely

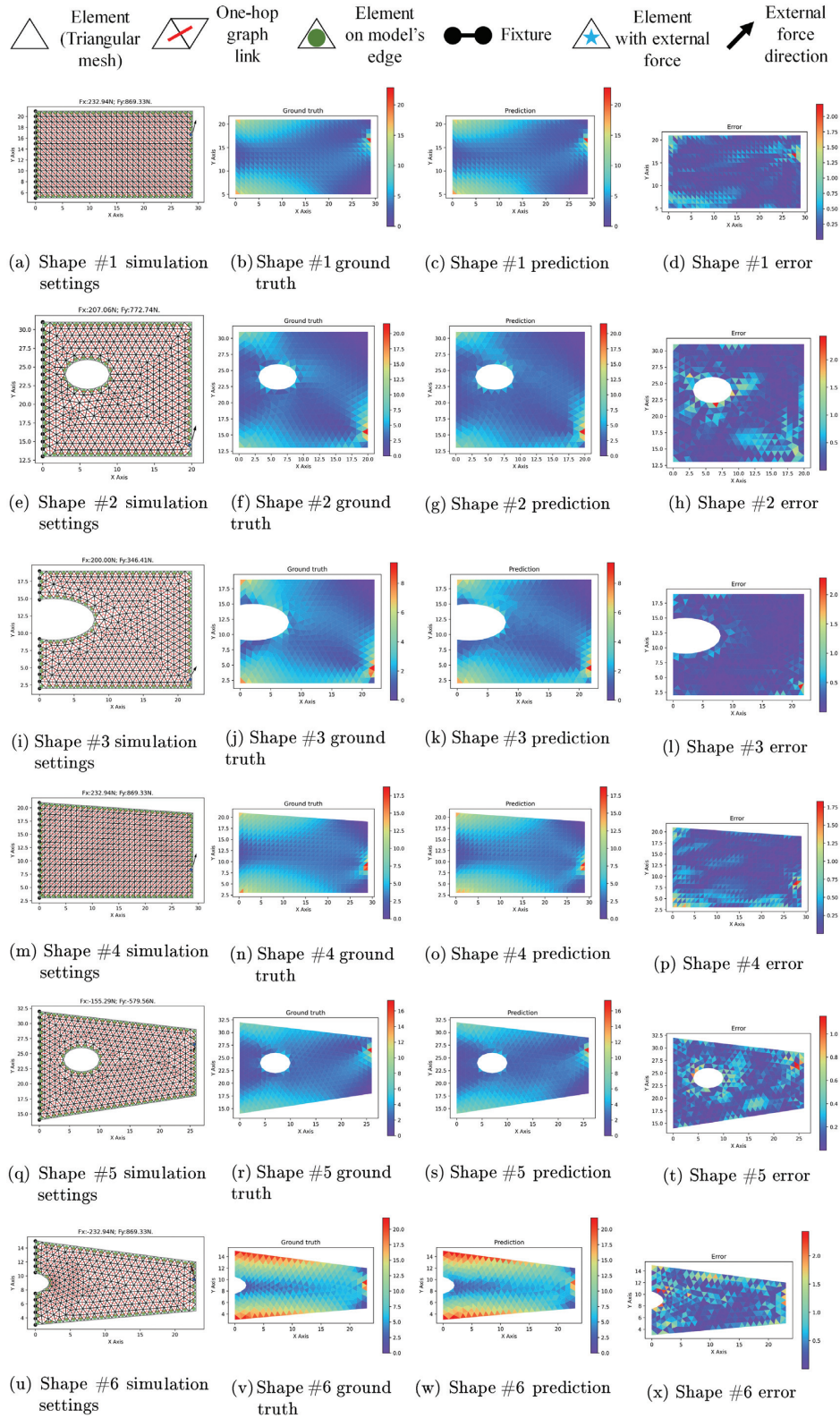


Figure 8: Prediction results for von Mises stress field.

improves the prediction accuracy and reaches 0.016383 testing accuracy with MAPE of 2.85%, which is accurate enough to serve as an efficient FEA surrogate prediction model. The testing results have been compared with BOGE with different adjacency matrices that encode local graph links with $l_e = 1, 2$. From Table 4, it can

be seen that additional local mesh information cannot improve the prediction accuracy. The testing accuracy for $l_e = 1$ (model #2 in Table 4) drops to 0.021737 with MAPE of 3.08% while that for $l_e = 2$ (model #3 in Table 4) drops to 0.027302 with MAPE of 3.44%. More local graph edges provide worse prediction accuracy,

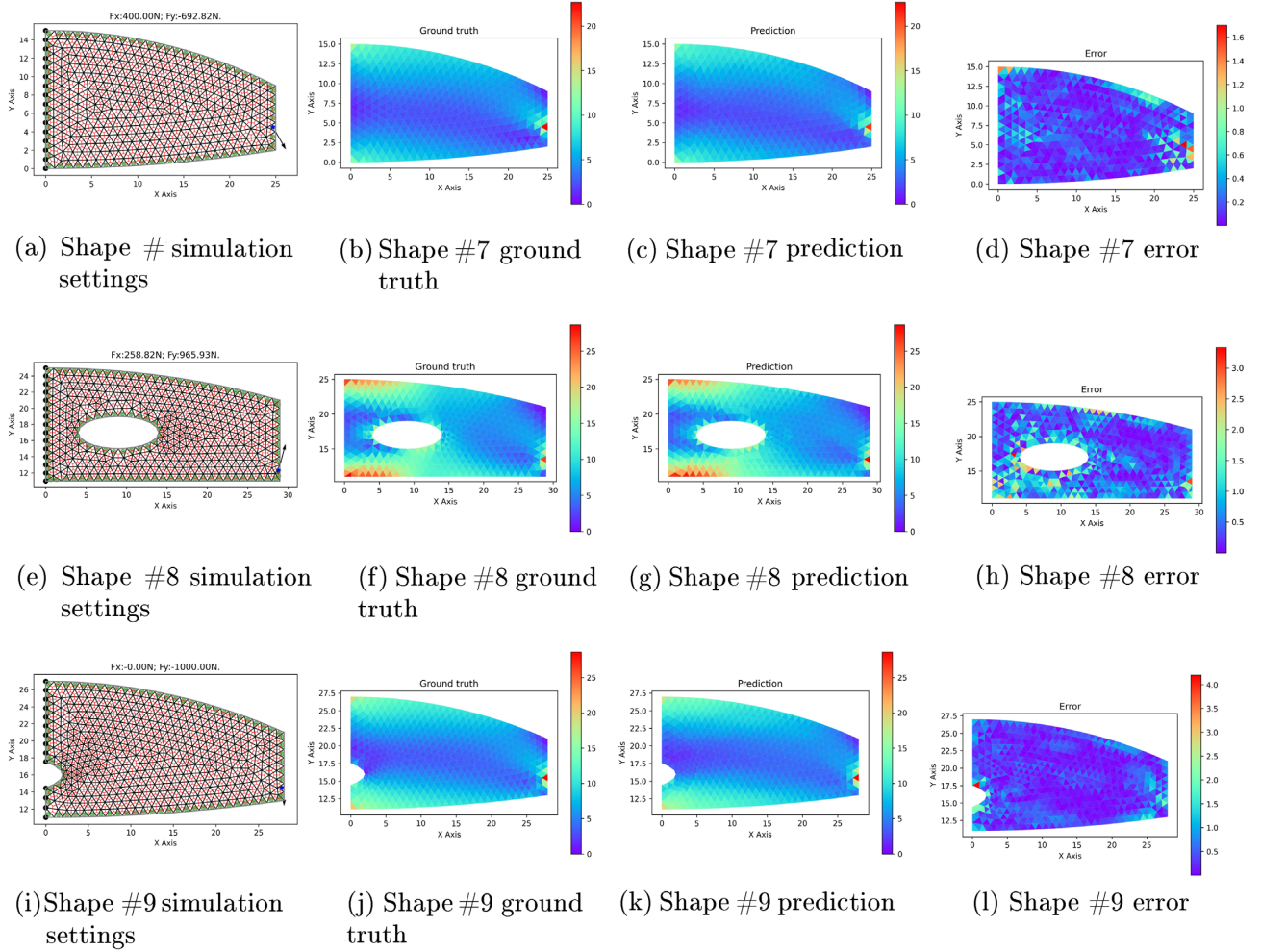


Figure 8 Continued.

Table 7: Analyses for topology optimization prediction results (training/validating/testing accuracy are presented without regularization).

No.	Models	Statistics for training / validating / testing subsets			
		MAPE	MAPE for $\{p p_i > 0.01\}$	Avg. number of meshes $\{p p_i > 0.01\}$	Avg. number of meshes $\{p \text{Abs.}(p_i - \hat{p}_i) > 0.01\}$
#1	DeepGCN(3 layer)	22.26% /	9.56% /	23.28% /	1.11% /
		22.65% /	9.71% /	23.87% /	1.49% /
		22.53%	9.70%	22.99%	1.58%
#2	DeepGCN(3 layer) with σ attached	21.35% /	10.21% /	23.62% /	6.23% /
		21.91% /	10.39% /	22.27% /	6.38% /
		21.57%	10.37%	23.49%	6.00%

which can be caused by message congestion (Loukas, 2019) since more elements enlarge the size of the message passing at each training epoch. Also, the adjacency matrix with more graph links can lead to a denser tensor product which takes more computational resources. Therefore, we only employ BOGE with $l_e = 0$ for the rest of the training task to provide a satisfying prediction result.

We also consider the effect of the dataset size. Another training experiment with 180k simulation dataset ("Stress prediction (180k)" in Table 2) is generated by the same 3-layer DeepGCN structure (model #4 in Table 4). The testing accuracy increases

to 0.011706 which shows a better training performance, but only slightly improves the MAPE which reaches 2.41%. This indicates that while the larger training dataset provides better results, a proper smaller dataset with around 45k data can still provide appropriate training results. The training loss and the validation/testing accuracy for the stress predictions are shown in Fig. 7a. Some of the predicted results from model #4 in Table 4 are presented in Fig. 8 which illustrates the effectiveness of the BOGE approach. The average prediction time for the GNN model (model #4 in Table 4) with GPU is 10 ms which is far less than the ABAQUS computation time (around 0.947 s in Table 2).

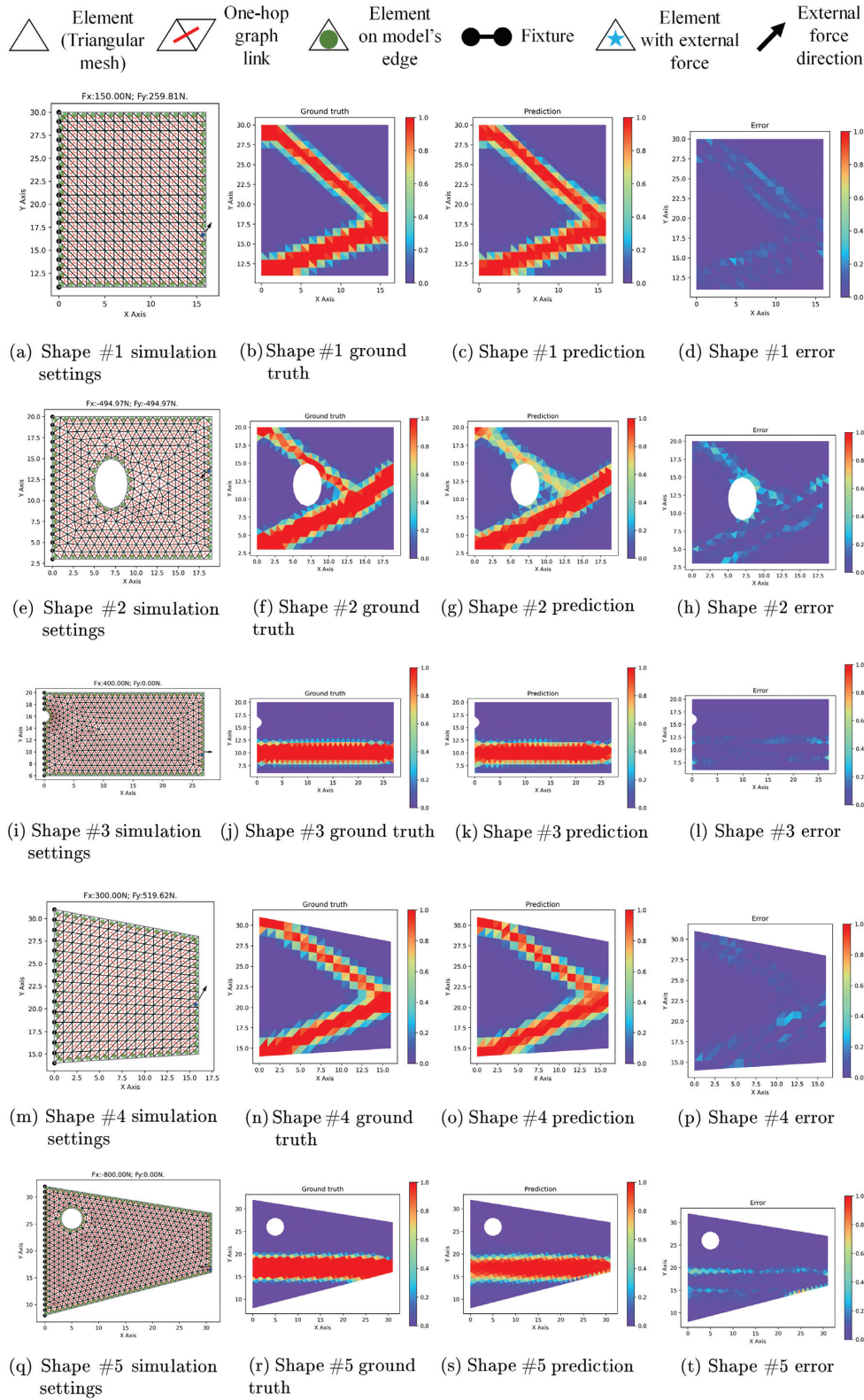


Figure 9: Prediction results for topology optimizations.

4.3. Stress field prediction with BOGE for the displacement boundary condition and the generalization of the work

We follow the best model in Table 4 (model #1 and #4) to fit the stress field distribution of “Stress prediction – displacement

BC (45k)” dataset in Table 2 using the BOGE with $l_e = 0$. The result shows that the MSE drops to 0.013486 with MAPE of 2.07%, shown in Table 3. This validates that, with the BOGE approach, the GNN can predict the stress field with a displacement boundary condition. The prediction time for the GNN is

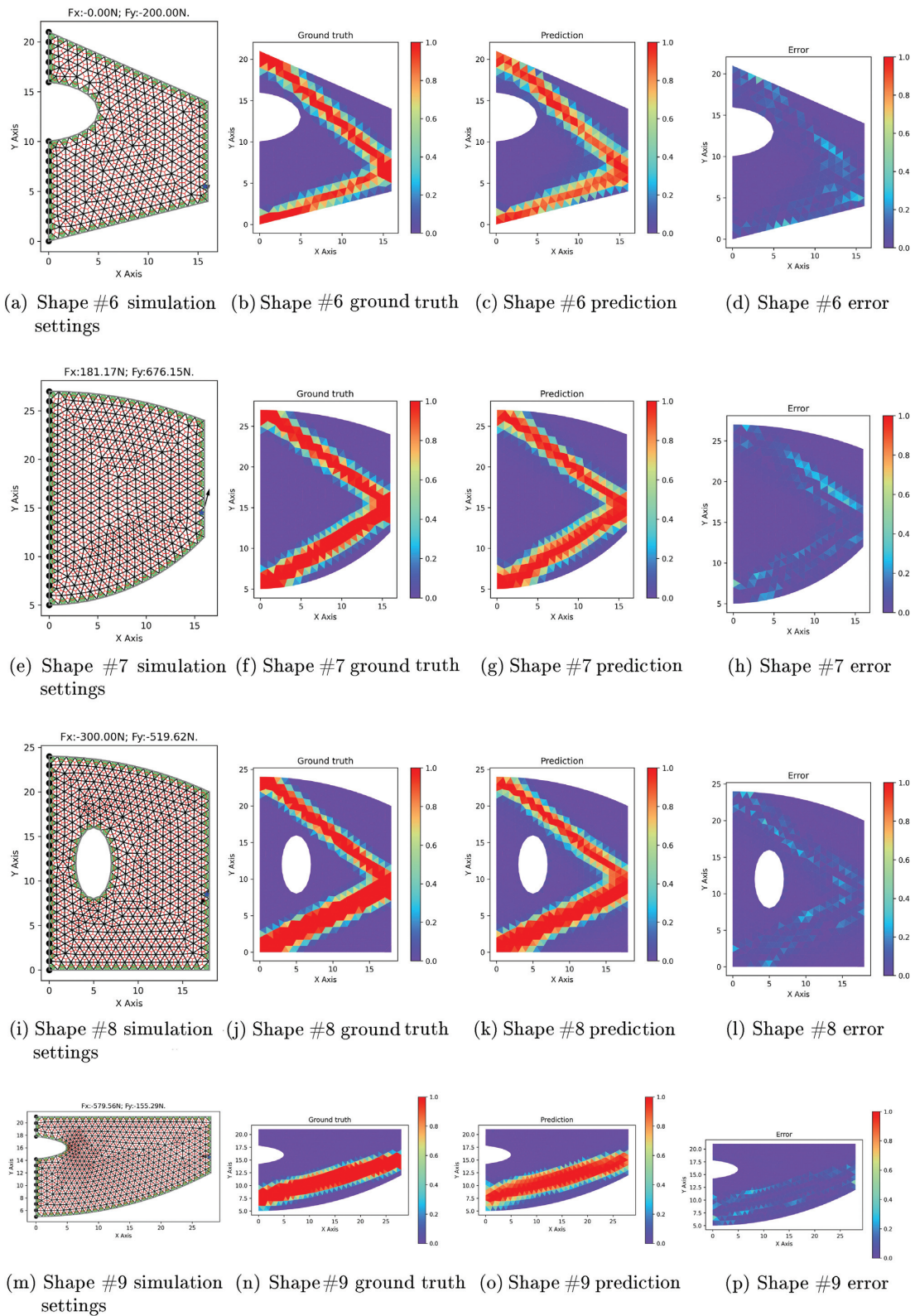


Figure 9 Continued.

11 ms whose magnitude is one order smaller than the ABAQUS simulation.

The generalization of this method can be questioned since there are some other boundary conditions in the cantilever beam problem, e.g., surface traction, body load, and combined condi-

tions. Demonstrations only on the point load and the displacement boundary conditions may not be sufficient to guarantee that our method can work on other solid mechanics problems. However, in order to avoid making too many validation experiments for the cost of the training can be unaffordable, we consider the su-

perposition since the cantilever beam problem is a linear problem if only the elastic condition is considered. The deep neural network is also capable of fitting the superposition problem according to the universal approximation theory (Hornik et al., 1989). Therefore, if other boundary conditions can be represented by the combination of the point load and the displacement boundary conditions, we can state that this method has the potential to serve as an interpolation method to other linear solid mechanics problems.

First, since the computation in FEA is generally executed on mesh nodes, the surface traction or the body load are usually decomposed into the point loads and the point displacements. These boundary conditions can be represented by the superposition of our dataset. Further, the combination of surface traction, point load, surface displacement, and point displacement can also be represented by the superposition of the separated boundary conditions. Therefore, GNN with BOGE has the potential to solve the solid mechanics problem under complex boundary conditions.

Last, the generalization of the DL model is still under research. Compared with other works (Guo & Buehler, 2020; Guo et al., 2016), we provide sufficient evidence to validate that the BOGE method with GNN can work for the cantilever beam problem, and it has the potential to solve other solid mechanics problems. The value of the DL surrogate model is more for the application, and some of the state-of-the-art DL-based surrogate models (Belbute-Peres et al., 2020; Khadiolkar et al., 2019; Lee et al., 2020; Nie et al., 2020, 2021; Ogoke et al., 2020; Wang et al., 2020; Zhang et al., 2018a) only consider one type of the boundary condition or one type of application with the same boundary conditions and their work is still valuable according to the merit of their applications. A fast cantilever-beam interpolation method is helpful and the BOGE approach with GNN can pave the way for solving other more complex boundary condition problems with unstructured meshes. Further, the other more significant objective of using the deep neural network is to solve the design problem which requires highly abstract decision-making capabilities. These design strategies cannot be simply represented by the conventional methods, but can relatively easily be fitted by the deep neural network. Therefore, in the next section, we will present the prediction result on topological optimization to validate this important application.

4.4. Topology optimization prediction

Using the same method, we regress the topology optimization results to investigate the BOGE's potential on solving abstract decision-making problems and provide an end-to-end design optimization solution. Using the same DeepGCN structure, we employ the same graph embedding inputs to regress the volume density z_e . Since topology optimization usually happens after the stress field simulation, we also consider the condition when the stress field information is added to the features of graph vertices. In this condition, the stress field information $\sigma = [\sigma_x, \sigma_y, \tau_{xy}]^T$ is attached to the graph-vertex features in BOGE approach. The training data employs BOGE with $l_e = 1$ on the 45k topology optimization dataset ("Topology optimization (45k)" in Table 2), and its training results are shown in Table 5. The dropout layers have been added after each layer with a dropout rate of 0.1 to provide a better prediction result.

From the training results, it can be seen that BOGE without encoding the stress information (model #1 in Table 5) reaches 0.002735 testing accuracy, which is better than the best testing MSE – 0.059943 in Nie et al. (2021) with similar cantilever beam

settings and the comparable number of meshes. However, the MAPE is relatively large and reaches 22.53% in testing accuracy. It is due to two effects. First, the range or the average value for the volume density in the topological optimization is relatively small. For stress prediction shown in Table 2, the average von Mises stress is around 2 MPa while the maximum stress magnitude can reach 74.09 MPa, which means the error divided by its ground truth value can be relatively small for the MAPE. Assuming one element with the stress of 2 MPa (which is the average stress) has a prediction error of 0.1 MPa, the MAPE for this element is only 5%. However, from Table 2, the average volume density is around 0.29 whereas the maximum volume density is 1.0, which means that the same error can contribute largely to the final MAPE. For example, if the element with 0.29 volume density has a prediction error of 0.1, the MAPE for this element can be 34%. This explains one of the reasons that some researchers have not employed the MAPE for evaluating topology optimization results (Banga et al., 2018; Nie et al., 2021; Zhang et al., 2019). Further, the large MAPE is generated because some outliers around the edge of the optimized beam contribute to most of the errors. Assume that the resolution of the volume density is 0.01, which is also the ϵ in equation (11). From model #1 in Table 7, when only meshes with its ground-truth volume density larger than ϵ are considered ($p|p_i > 0.01$), the MAPE for those meshes is only 9.70%. Also, for each testing graph (or simulation model), around 22–23% of its meshes have a volume density larger than 0.01 for its ground truth value, but only 1.58% of the total meshes contribute to the errors larger than ϵ . Some prediction results from model #1 in Table 5 are shown in Fig. 9. It can be seen that this 1.58% of meshes mostly are located primarily on the edges of the optimized beams. Since the boundary of the optimized beams usually has a volume density around $\epsilon = 0.01$ as its ground truth value, any small prediction error can be largely amplified by the MAPE. However, regardless of the MAPE, the predicted volume density shown in Fig. 9 and its testing accuracy are sufficient to validate the BOGE's advantage in regressing complex decision-making problems.

With the stress field information, BOGE shows a worse prediction result, which reaches the MSE of 0.003378 for testing accuracy (model #2 in Table 5). Also, more outliers (6.00% from model #2 in Table 7) are found in this condition. Adding additional features to the graph vertex can lead to message congestion (Loukas, 2019) and oversquashing (Alon & Yahav, 2020). Therefore, unlike the CNN-based surrogate model (Nie et al., 2021), GNN models are sensitive to the size of the input tensor and cannot be simply enhanced by adding extra input features. Balancing the input feature, hidden layer features, and the GNN layers can provide better results according to the GNN's property.

The topology optimization prediction shows BOGE's capability of making abstract decisions which benefits the smart design technology. The average prediction time for the topology-optimization GNN model (model #1 in Table 5) on GPU is around 10 ms, similar to that of GNN stress field prediction, but far less than the ABAQUS computation time (around 62 s in Table 2). This is because the SIMP method requires several iterations of computations that take much longer than the stress prediction. However, the GNN surrogate model can directly output the optimized result which largely saves time costs. The overall performance of BOGE demonstrates its irreplaceable advantage in regressing the physical field of unstructured elements. The training loss and the validation/testing accuracy for the stress predictions is shown in Fig. 7b.

5. Conclusions

We develop a BOGE approach for the GNN-based FEA surrogate model to solve the cantilever beam problem with triangular meshes. The stress field has been generated and the optimized design is provided efficiently within around 10 ms. The BOGE approach bypasses the limitations of the MPNN framework and provides shortcuts for both global boundary information and local multi-hop elements, which allows shallow-layer GNN to regress boundary value problems with long-range graph-vertex interactions. Working on solving the cantilever beam problem, the BOGE approach with 3-layer DeepGCN model achieves the regression with MSE of 0.011 706 (2.41% MAPE) for stress field prediction and 0.002 735 MSE (with 1.58% elements having error larger than 0.01) for topological optimization, which validates the BOGE's efficiency in complex decision-making problems. The model has the potential to be applied to most physical-field fitting problems with any type of polygon meshes or mixed types of meshes. It can largely improve the efficiency of the digital prototyping process and contribute to the computational design field. This study has potential limitations. First, a fairer comparison between the conventional algorithm and the DL method should be conducted in the future, since currently, the conventional algorithm runs on a serial CPU while the GNN method uses a parallel GPU. Future work can focus on developing the solid mechanics solver based on Pytorch with GPU acceleration to make a better baseline for the computation time. Further, the GNN performance is sensitive to the input-tensor shape and requires more theoretical analysis to investigate appropriate methods to improve the structure of GNN. Future work can focus on modifying GNN structures to obtain more accurate regression results and applying the BOGE approach to other boundary value problems. Last, the parametric dataset needs to be expanded to include more cantilever beam geometries with different boundary conditions and different mesh distributions to test the validation of the BOGE approach. Other real solid mechanics applications can be implemented in the future to testify BOGE's performance.

Acknowledgments

This work was supported in part by the Technology Innovation Program (20015060, Development of Hybrid 3D Printing Machine for Large Scale Additive Manufacturing and Machining Process of CFRP Lightweight Parts) funded By the Ministry of Trade, Industry & Energy (MOTIE, Korea). This work was also supported in part by the Clean Energy Smart Manufacturing Innovation Institute (CESMII) funded by the US Department of Energy (DOE Federal Award, No: DE-EE0007613).

Conflict of interest statement

None declared.

References

- Abaqus, G. (2011). *Abaqus 6.11*. Dassault Systemes Simulia Corporation.
- Alet, F., Jeewajee, A. K., Villalonga, M. B., Rodriguez, A., Lozano-Perez, T., & Kaelbling, L. (2019). Graph element networks: Adaptive, structured computation and memory. In *Proceedings of the International Conference on Machine Learning* (pp. 212–222). PMLR.
- Alon, U., & Yahav, E. (2020). On the bottleneck of graph neural networks and its practical implications. arXiv preprint arXiv:2006.05205. <https://doi.org/10.48550/arXiv.2006.05205>.
- Axelsson, O., & Barker, V. A. (2001). *Finite element solution of boundary value problems: Theory and computation*. SIAM.
- Banga, S., Gehani, H., Bhilare, S., Patel, S., & Kara, L. (2018). 3D topology optimization using convolutional neural networks. arXiv preprint arXiv:1808.07440. <https://doi.org/10.48550/arXiv.2006.05205>.
- Belbute-Peres, F. D. A., Economou, T., & Kolter, Z. (2020). Combining differentiable PDE solvers and graph neural networks for fluid flow prediction. In *Proceedings of the International Conference on Machine Learning* (pp. 2402–2411). PMLR.
- Capuano, G., & Rimoli, J. J. (2019). Smart finite elements: A novel machine learning application. *Computer Methods in Applied Mechanics and Engineering*, **345**, 363–381. <https://doi.org/10.1016/j.cma.2018.10.046>.
- Courtecuisse, H., Allard, J., Kerfriden, P., Bordas, S. P., Cotin, S., & Duriez, C. (2014). Real-time simulation of contact and cutting of heterogeneous soft-tissues. *Medical image analysis*, **18**, 394–410. <https://doi.org/10.1016/j.media.2013.11.001>.
- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with pytorch geometric. arXiv preprint arXiv:1903.02428. <https://doi.org/10.48550/arXiv.1903.02428>.
- Fu, X., Peddireddy, D., Aggarwal, V., & Jun, M. B. -G. (2021). Improved dixel representation: A 3d CNN geometry descriptor for manufacturing CAD. *IEEE Transactions on Industrial Informatics*, **18**, 5882–5892. <https://doi.org/10.1109/TII.2021.3136167>.
- Gao, H., & Ji, S. (2019). Graph U-Nets. In *Proceedings of the International Conference on Machine Learning* (pp. 2083–2092). PMLR.
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *Proceedings of the International Conference on Machine Learning* (pp. 1263–1272). PMLR.
- Guo, K., & Buehler, M. J. (2020). A semi-supervised approach to architected materials design using graph neural networks. *Extreme Mechanics Letters*, **41**, 101029. <https://doi.org/10.1016/j.eml.2020.101029>.
- Guo, X., Li, W., & Iorio, F. (2016). Convolutional neural networks for steady flow approximation. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 481–490).
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778).
- Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, **2**, 359–366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8).
- Kantzos, C., Lao, J., & Rollett, A. (2019). Design of an interpretable convolutional neural network for stress concentration prediction in rough surfaces. *Materials Characterization*, **158**, 109961. <https://doi.org/10.1016/j.matchar.2019.109961>.
- Khadilkar, A., Wang, J., & Rai, R. (2019). Deep learning-based stress prediction for bottom-up SLA 3D printing process. *The International Journal of Advanced Manufacturing Technology*, **102**, 2555–2569. <https://doi.org/10.1007/s00170-019-03363-4>.
- Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907. <https://doi.org/10.48550/arXiv.1609.02907>.
- Koeppel, A., Bamer, F., & Markert, B. (2020). An intelligent nonlinear meta element for elastoplastic continua: Deep learning using a new time-distributed residual U-Net architecture. *Computer Methods in Applied Mechanics and Engineering*, **366**, 113088. <https://doi.org/10.1016/j.cma.2020.113088>.
- Krokos, V., Bui Xuan, V., Bordas, S., Young, P., & Kerfriden, P. (2022). A Bayesian multiscale CNN framework to predict local stress fields in structures with microscale features. *Computational*

- Mechanics, **69**, 733–766. <https://doi.org/10.1007/s00466-021-02112-3>.
- Kutz, J. N. (2017). Deep learning in fluid dynamics. *Journal of Fluid Mechanics*, **814**, 1–4. <https://doi.org/10.1017/jfm.2016.803>.
- Lee, H., Lee, J., Kim, H., & Mun, D. (2022). Dataset and method for deep learning-based reconstruction of 3D CAD models containing machining features for mechanical parts. *Journal of Computational Design and Engineering*, **9**, 114–127. <https://doi.org/10.1093/jcde/qwab072>.
- Lee, S., Kim, H., Lieu, Q. X., & Lee, J. (2020). CNN-based image recognition for topology optimization. *Knowledge-Based Systems*, **198**, 105887. <https://doi.org/10.1016/j.knsys.2020.105887>.
- Li, G., Muller, M., Thabet, A., & Ghanem, B. (2019). DeepGCNs: Can GCNs go as deep as CNNs?. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (pp. 9267–9276).
- Li, G., Xiong, C., Thabet, A., & Ghanem, B. (2020). DeeperGCN: All you need to train deeper GCNs. arXiv preprint arXiv:2006.07739. <https://doi.org/10.48550/arXiv.2006.07739>.
- Loukas, A. (2019). What graph neural networks cannot learn: Depth vs width. arXiv preprint arXiv:1907.03199. <https://doi.org/10.48550/arXiv.1907.03199>.
- Nie, Z., Jiang, H., & Kara, L. B. (2020). Stress field prediction in cantilevered structures using convolutional neural networks. *Journal of Computing and Information Science in Engineering*, **20**, 011002. <https://doi.org/10.1115/1.4044097>.
- Nie, Z., Lin, T., Jiang, H., & Kara, L. B. (2021). TopologyGAN: Topology optimization using generative adversarial networks based on physical fields over the initial domain. *Journal of Mechanical Design*, **143**, 031715. <https://doi.org/10.1115/1.4049533>.
- Ogoke, F., Meidani, K., Hashemi, A., & Farimani, A. B. (2020). Graph convolutional neural networks for body force prediction. arXiv preprint arXiv:2012.02232. <https://doi.org/10.48550/arXiv.2012.02232>.
- Peddireddy, D., Fu, X., Shankar, A., Wang, H., Joun, B. G., Aggarwal, V., Sutherland, J. W., & Jun, M. B.-G. (2021). Identifying manufacturability and machining processes using deep 3D convolutional networks. *Journal of Manufacturing Processes*, **64**, 1336–1348. <https://doi.org/10.1016/j.jmapro.2021.02.034>.
- Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., & Battaglia, P. W. (2020). Learning mesh-based simulation with graph networks. arXiv preprint arXiv:2010.03409. <https://doi.org/10.48550/arXiv.2010.03409>.
- Rong, Y., Huang, W., Xu, T., & Huang, J. (2019). DropEdge: Towards deep graph convolutional networks on node classification. arXiv preprint arXiv:1907.10903. <https://doi.org/10.48550/arXiv.1907.10903>.
- Roy, A., Nabi, M., & Rahman, N. (2021). Finite element compatible matrix interpolation for parametric model order reduction of electrothermal microgripper. *Journal of Computational Design and Engineering*, **8**, 1622–1635. <https://doi.org/10.1093/jcde/qwab066>.
- Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J., & Battaglia, P. (2020). Learning to simulate complex physics with graph networks. In *Proceedings of the International Conference on Machine Learning* (pp. 8459–8468). PMLR.
- Tamaddon-Jahromi, H. R., Chakshu, N. K., Sazonov, I., Evans, L. M., Thomas, H., & Nithiarasu, P. (2020). Data-driven inverse modelling through neural network (deep learning) and computational heat transfer. *Computer Methods in Applied Mechanics and Engineering*, **369**, 113217. <https://doi.org/10.1016/j.cma.2020.113217>.
- Tepjit, S., Horváth, I., & Rusák, Z. (2019). The state of framework development for implementing reasoning mechanisms in smart cyber-physical systems: A literature review. *Journal of Computational Design and Engineering*, **6**, 527–541. <https://doi.org/10.1016/j.jcde.2019.04.002>.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. arXiv preprint arXiv:1710.10903. <https://openreview.net/forum?id=rjXmpikCZ>.
- Wang, D., Xiang, C., Pan, Y., Chen, A., Zhou, X., & Zhang, Y. (2019). A deep convolutional neural network for topology optimization with strong generalization ability. *Engineering Optimization*, **54**, 973–988. <https://doi.org/10.1080/0305215X.2021.1902998>.
- Wang, R., Zhou, X., Dong, L., Wen, Y., Tan, R., Chen, L., Wang, G., & Zeng, F. (2020). Kalibre: Knowledge-based neural surrogate model calibration for data center digital twins. In *Proceedings of the 7th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation* (pp. 200–209).
- Yang, C., Wang, R., Yao, S., Liu, S., & Abdelzaher, T. (2020). Revisiting “over-smoothing” in deep GCNs. arXiv preprint arXiv:2003.13663. <https://doi.org/10.48550/arXiv.2003.13663>.
- Zhang, Y., Sung, W. J., & Mavris, D. N. (2018a). Application of convolutional neural network to predict airfoil lift coefficient. In *Proceedings of the 2018 AIAA/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference* (p. 1903).
- Zhang, Z., Jaiswal, P., & Rai, R. (2018b). FeatureNet: Machining feature recognition based on 3D convolution neural network. *Computer-Aided Design*, **101**, 12–22. <https://doi.org/10.1016/j.cad.2018.03.006>.